MINI PROJECT REPORT

AI Dataset Cleaning & Validation Tool

Detecting missing values, duplicates and inconsistent formats in a CSV using Python, Gemini and prompt engineering

Subject	AI Tools and Prompt Engineering
Project category	Data Processing
Project topic	AI Dataset Cleaning and Validation Tool
Practical task	Detect missing values, duplicates and inconsistent formats in a CSV
Expected output	Cleaned dataset and validation report

Submitted by	Tanmay Vijay Sherkar
Enrollment / roll no.	150096725139
Faculty guide	Prof.Poonam
Date of submission	2/11/2026
Project repository	Github

Declaration

I, Tanmay Sherkar, declare that this mini project report titled **AI Dataset Cleaning and Validation Tool** is my own work, carried out as part of the subject AI Tools and Prompt Engineering. AI tools were used to help write code, and every prompt I used, along with what the AI returned and what I changed, is documented in Section 5 and Appendix A. All data used is synthetic and contains no real personal information.

Date: 2 November 2026

Contents

Abstract 4
1. Introduction 5
2. Tools and Technologies 6
3. System Design 8
4. Dataset 10
5. Prompt Engineering 12
6. Implementation and Screenshots 31
7. Challenges and Limitations 36
8. Conclusion and Future Work 37
References 37
Appendix A: Build prompt log 38
Appendix B: Links and files 38

Abstract

Real-world CSV files are rarely clean. They contain blank or disguised missing values such as N/A and a dash, duplicate and near-duplicate records, and the same information written in several formats, for example dates as 2025-03-04, 04/03/2025 and 4 Mar 2025. This project builds a Streamlit tool that detects these problems, fixes the safe ones, flags the rest for a human, and produces a cleaned dataset together with a validation report.

The key design decision is a split of responsibility: deterministic Python code detects and edits data, while the Gemini API only suggests rules (formats, category mappings, handling strategies) and explains results in plain English. Every Gemini answer is returned as JSON that follows a fixed schema, and no rule is applied until the user approves it. The tool was tested on a synthetic e-commerce orders file of 545 rows with 848 planted problems whose exact locations are known, so detection and cleaning can be measured rather than judged by eye.

The project was built using AI tools and prompt engineering throughout. Eight build prompts were used to generate the code and five prompts run inside the tool; each was tested, scored and revised across versions.

Keywords: data cleaning, data validation, prompt engineering, Gemini API, pandas, Pandera, Streamlit.

1. Introduction

1.1 Background

Data cleaning is a large share of any data project, and poor data quality leads directly to wrong analysis. Spreadsheet-style CSV files are especially error-prone because they have no enforced types: any cell can hold any text. Large language models can read column names and sample values and recognise, for example, that COD and Cash on Delivery are the same payment method, which rule-based tools struggle with. They can also make confident mistakes, so they need guardrails.

1.2 Problem statement

Given an arbitrary CSV file, automatically detect missing values, duplicate rows and inconsistent formats, apply safe corrections, and report exactly what was found and changed, using AI where it adds value and without letting the AI silently alter data.

1.3 Objectives

- Detect blank and disguised missing values in every column.
- Detect exact duplicates, repeated keys with conflicting content, and near-duplicate records.
- Detect inconsistent formats in dates, phone numbers, emails, currency amounts, casing and category labels.
- Use the Gemini API to propose cleaning rules as structured JSON, with a human approval step.
- Produce a cleaned CSV, an HTML validation report and a change log.
- Document the prompt engineering process: every prompt, its versions, scores and failures.

1.4 Scope and limitations

The tool handles one CSV file at a time of up to roughly 100,000 rows. It uses the free tier of the Gemini API, so it is designed to make fewer than 40 model calls per file and to cache results. Only synthetic data is sent to the API, because content sent on the free tier may be used by Google to improve its products. Dates are assumed to be day-first (DD/MM).

2. Tools and Technologies

Layer	Tool	Why it was chosen
Language	Python 3.11+	Standard for data work
Data handling	pandas	Simple and familiar; enough for CSVs of this size
Schema validation	Pandera	Lightweight; schema-based checks with zero configuration
Fuzzy duplicates	RapidFuzz	Fast string similarity scores for near-duplicate names
Date parsing	python-dateutil	Parses mixed date formats
Phone checks	phonenumbers	Validates numbers per country code
LLM	Gemini via google-genai SDK	Free tier and JSON output mode
Output schemas	Pydantic	Forces Gemini answers into a fixed shape
User interface	Streamlit	Upload, review and download in one page
Report	Jinja2 + HTML	Fully custom validation report
Testing	pytest	Tests run against the planted answer key
Version control	Git + GitHub	Commit history is evidence of the build
AI coding assistant	Antigravity	Wrote code from the build prompts B1 to B8

2.1 Gemini API setup

The model used was gemini-3.8-flash through the google-genai SDK with the key stored in an environment variable (never committed to Git). A one-line test call confirmed the key and model worked before any development started.

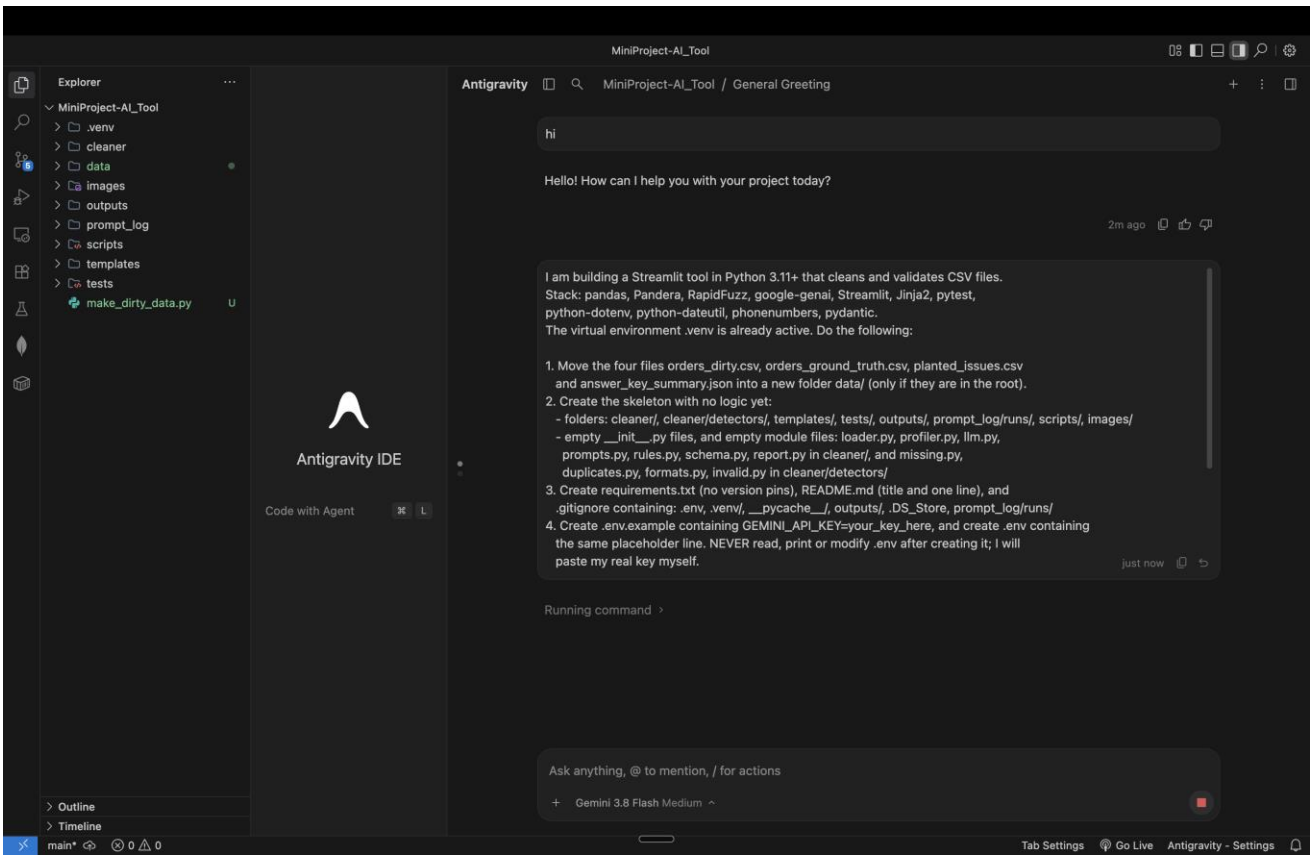


Figure 1: *Google AI Studio page showing the API key created (hide the key itself)*

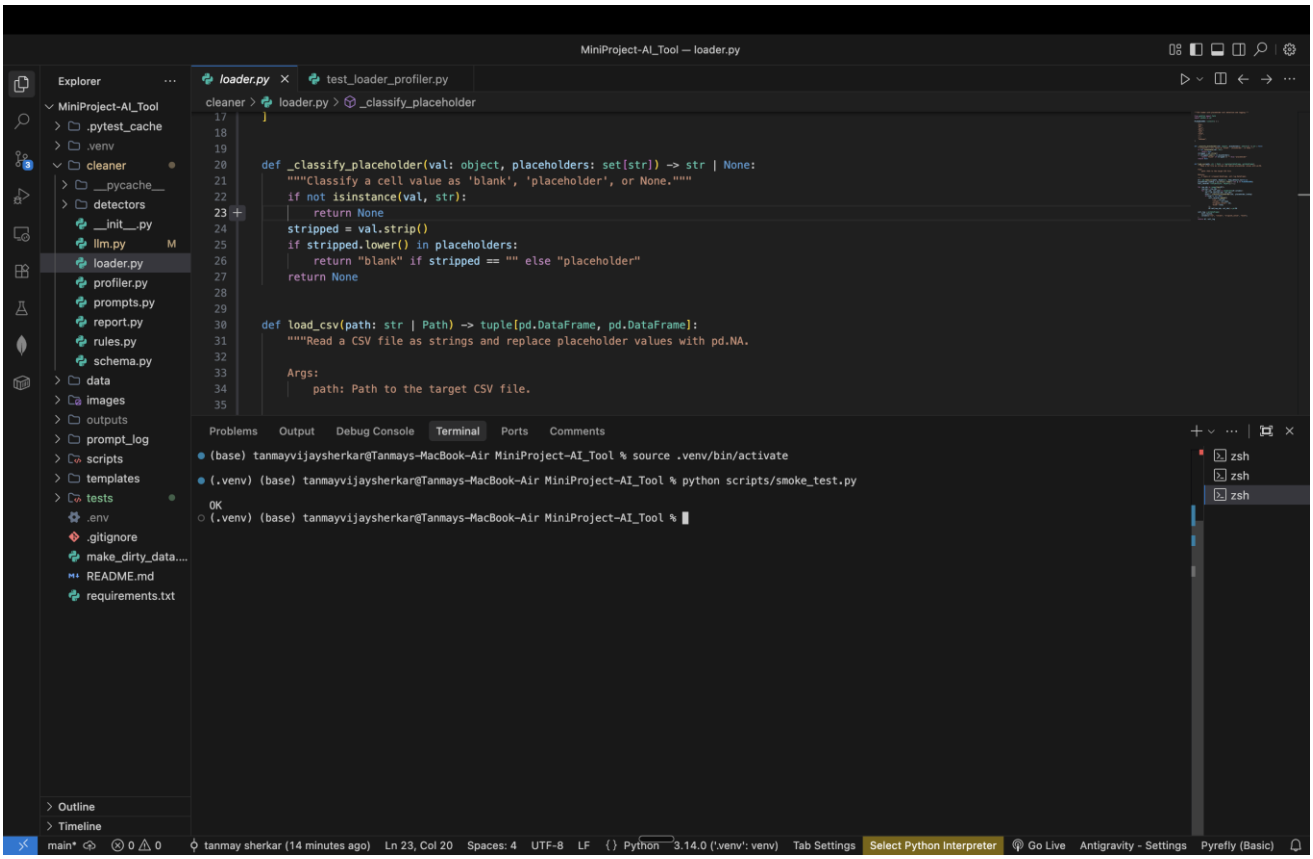


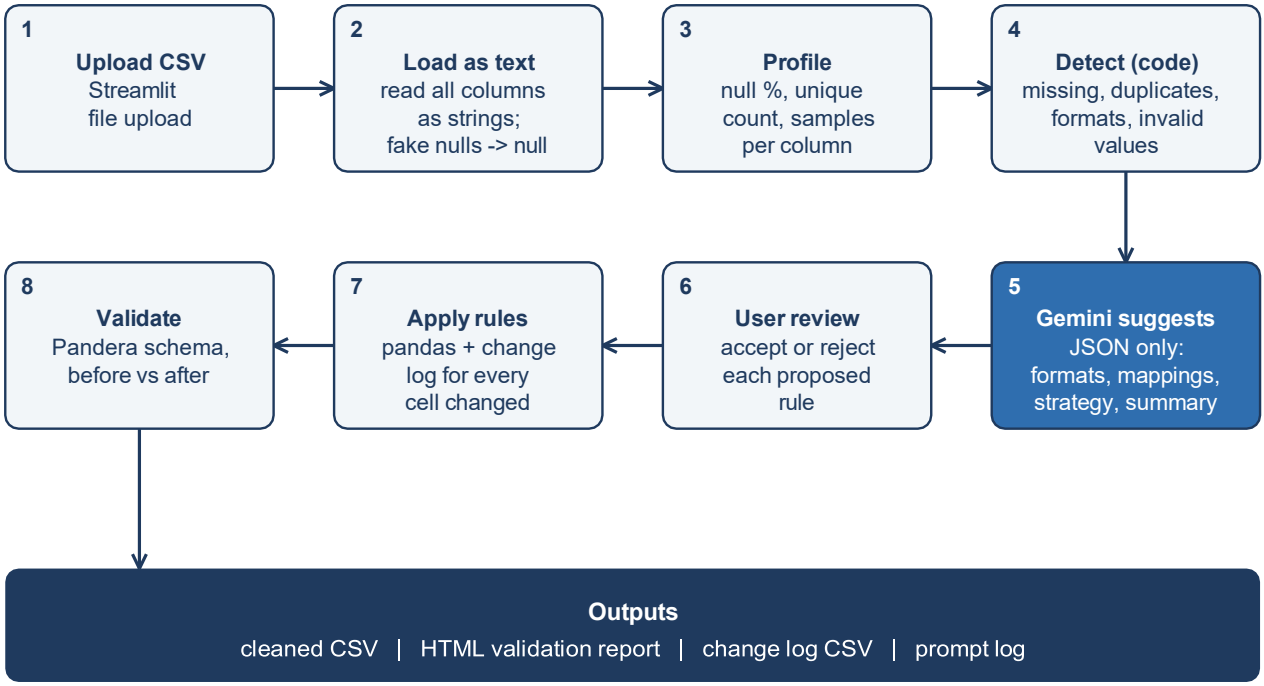
Figure 2: *Smoke test: code and the printed reply from the Gemini API*

3. System Design

3.1 Design principle

Code detects and edits; Gemini only suggests and explains. Detection by code is exact and repeatable. Gemini is used where meaning matters, such as deciding that Bombay and Mumbai are the same city, and its answers are validated against a schema and approved by the user before anything changes. Gemini never fills a missing value with a guess.

3.2 Pipeline



Processing pipeline. Steps 1 to 4 run in code; step 5 (Gemini) only proposes rules; steps 7 and 8 run only after the user approves in step 6.

3.3 Cleaning rules

Issue	How it is detected	What the tool does
Blank or fake nulls (N/A, -, ?)	Placeholder list on every column	Convert to real null and flag
Missing key value	Null in id, date, price, quantity	Flag; fill only with an approved rule
Blank rating on unfinished order	Null rating where status is not Delivered or Returned	Leave alone: this blank is legitimate
Exact duplicate row	df.duplicated()	Keep first, drop the rest, log them
Reused order id, different content	Key appears twice, rows differ	Flag for human review
Near-duplicate (name typo, case)	RapidFuzz score 90+ among rows sharing customer id and date	Show the pair; user confirms the drop
Mixed date formats	Pattern detection, then dateutil with day first	Convert to YYYY-MM-DD
Impossible or future date	Parse failure, or later than today	Flag; set null only if approved

Issue	How it is detected	What the tool does
Phone variants, short numbers	phonenumbers with country from the row	Normalize; flag invalid
Invalid or uppercase email	Regex check	Lowercase valid ones; flag invalid
Money formats (Rs., INR, commas)	Pattern detection	Convert to plain number, 2 decimals
Category variants	Distinct values with counts	Apply approved mapping (Gemini P3)
Zero, negative or worded quantity	Not a positive integer	Flag; convert worded numbers only if approved
Rating outside 1 to 5	Range check	Flag; never edit
Total not equal to quantity x price	Recompute	Flag only: unknown which field is wrong
Price outlier	IQR rule within product category	Flag

4. Dataset

A synthetic Indian e-commerce orders file was generated with a Python script (random seed 42). It has 545 rows and 16 columns (order, customer, contact, date, location, product, quantity, price, payment, status and rating fields). A clean 500-row ground-truth copy and a log of every planted problem were produced at the same time, which allows detection recall and cleaning accuracy to be measured.

Problem group	Planted count	Examples
Missing values	129	Blanks and fake nulls (N/A, null, -, ?, nan, none, Unknown) across 8 columns
Duplicates	45 extra rows	25 exact copies, 15 near-duplicates, 5 reused order ids with different content
Format inconsistencies	618	Mixed date, phone and money formats; country, city, category, payment and status variants; casing and spaces
Invalid values and logic errors	56	Bad emails, ratings outside 1 to 5, zero or negative quantity, impossible and future dates, wrong totals, outliers
Total	**848**	

Two deliberate traps were included. First, 151 blank ratings are legitimate because those orders are pending, shipped or cancelled; a good tool must not flag them. Second, dates are day-first, so 04/05/2025 means 4 May 2025.

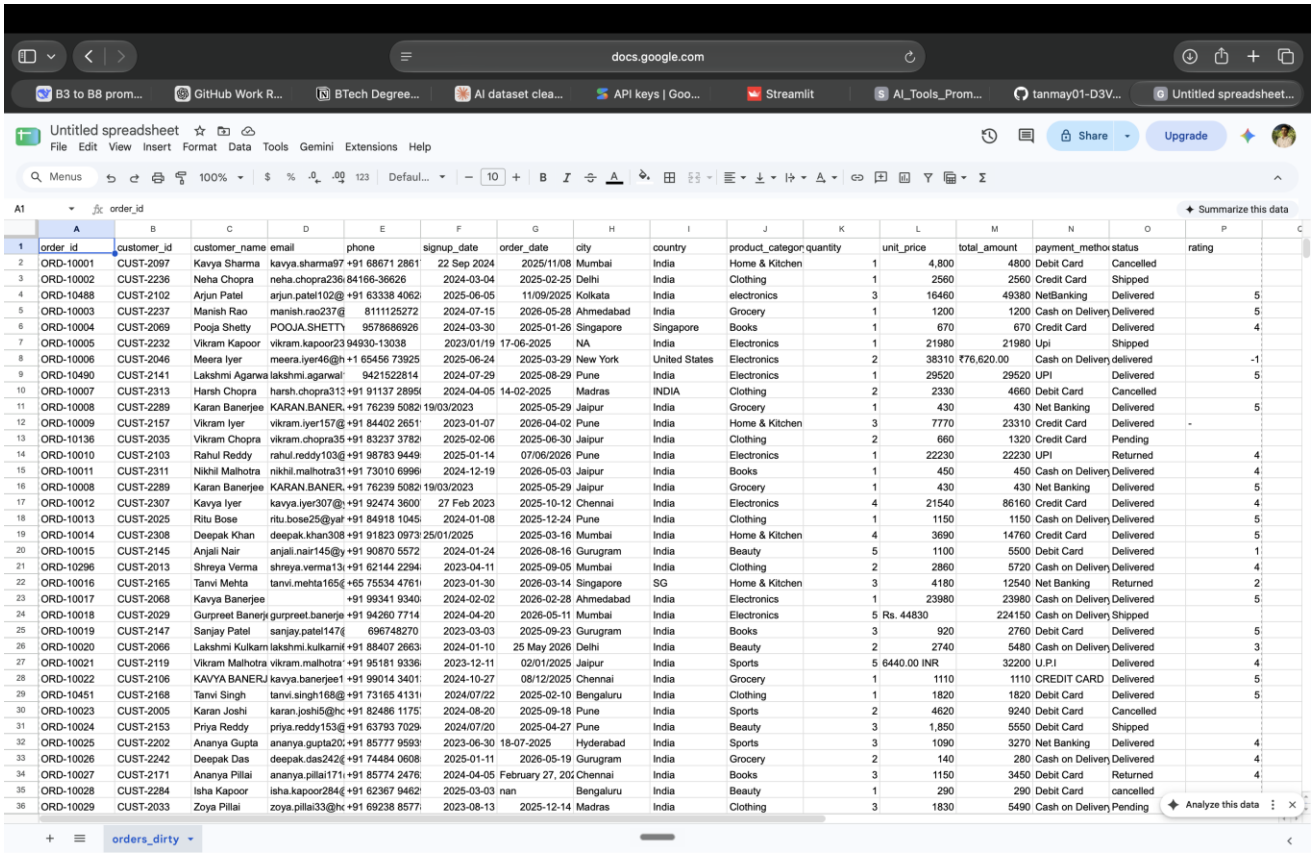
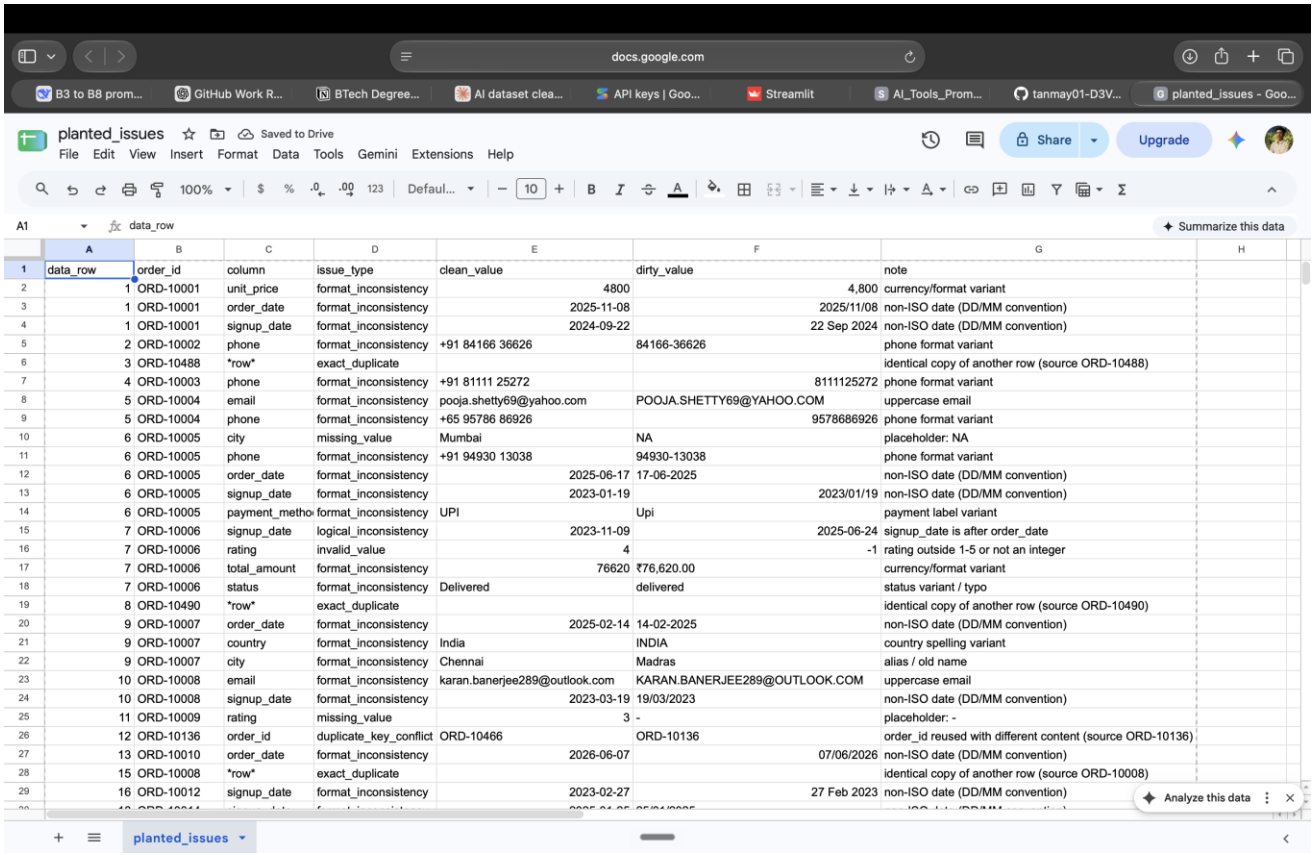


Figure 3: *A portion of orders_dirty.csv opened in a spreadsheet, showing visible mess*



	A	B	C	D	E	F	G	H
1	data_row	order_id	column	issue_type	clean_value	dirty_value	note	
2	1	ORD-10001	unit_price	format_inconsistency	4800	4,800	currency/format variant	
3	1	ORD-10001	order_date	format_inconsistency	2025-11-08	2025/11/08	non-ISO date (DD/MM convention)	
4	1	ORD-10001	signup_date	format_inconsistency	2024-09-22	22 Sep 2024	non-ISO date (DD/MM convention)	
5	2	ORD-10002	phone	format_inconsistency	+91 84166 36626	84166-36626	phone format variant	
6	3	ORD-10488	*row*	exact_duplicate			identical copy of another row (source ORD-10488)	
7	4	ORD-10003	phone	format_inconsistency	+91 81111 25272	8111125272	phone format variant	
8	5	ORD-10004	email	format_inconsistency	pooja.shetty69@yahoo.com	POOJA.SHETTY69@YAHOO.COM	uppercase email	
9	5	ORD-10004	phone	format_inconsistency	+65 95786 86926	9578686926	phone format variant	
10	6	ORD-10005	city	missing_value	Mumbai	NA	placeholder: NA	
11	6	ORD-10005	phone	format_inconsistency	+91 94930 13038	94930-13038	phone format variant	
12	6	ORD-10005	order_date	format_inconsistency	2025-06-17	17-06-2025	non-ISO date (DD/MM convention)	
13	6	ORD-10005	signup_date	format_inconsistency	2023-01-19	2023/01/19	non-ISO date (DD/MM convention)	
14	6	ORD-10005	payment_metho	format_inconsistency	UPI	Upi	payment label variant	
15	7	ORD-10006	signup_date	logical_inconsistency	2023-11-09	2025-06-24	signup_date is after order_date	
16	7	ORD-10006	rating	invalid_value	4	-1	rating outside 1-5 or not an integer	
17	7	ORD-10006	total_amount	format_inconsistency	76620	₹76,620.00	currency/format variant	
18	7	ORD-10006	status	format_inconsistency	Delivered	delivered	status variant / typo	
19	8	ORD-10490	*row*	exact_duplicate			identical copy of another row (source ORD-10490)	
20	9	ORD-10007	order_date	format_inconsistency	2025-02-14	14-02-2025	non-ISO date (DD/MM convention)	
21	9	ORD-10007	country	format_inconsistency	India	INDIA	country spelling variant	
22	9	ORD-10007	city	format_inconsistency	Chennai	Madras	alias / old name	
23	10	ORD-10008	email	format_inconsistency	karan.banerjee289@outlook.com	KARAN.BANERJEE289@OUTLOOK.COM	uppercase email	
24	10	ORD-10008	signup_date	format_inconsistency	2023-03-19	19/03/2023	non-ISO date (DD/MM convention)	
25	11	ORD-10009	rating	missing_value	3	-	placeholder: -	
26	12	ORD-10136	order_id	duplicate_key_conflict	ORD-10466	ORD-10136	order_id reused with different content (source ORD-10136)	
27	13	ORD-10010	order_date	format_inconsistency	2026-06-07	07/06/2026	non-ISO date (DD/MM convention)	
28	15	ORD-10008	*row*	exact_duplicate			identical copy of another row (source ORD-10008)	
29	16	ORD-10012	signup_date	format_inconsistency	2023-02-27	27 Feb 2023	non-ISO date (DD/MM convention)	

Figure 4: *A portion of planted_issues.csv (the answer key)*

5. Prompt Engineering

Prompt engineering appears twice in this project: **build prompts** (B1 to B8), which were given to an AI assistant to write the code, and **in-tool prompts** (P1 to P5), which run inside the application and call Gemini. Each prompt is logged with the exact text sent, what the AI returned, what was wrong, and what was changed.

5.1 Techniques used

Technique	Where	Example in this project
Role prompting	P1 to P5	"You are a careful data quality analyst."
Context and constraints	All	State the file, the columns and what must not change
Few-shot examples	P2, P3	Two or three input and output pairs inside the prompt
Structured output	P1 to P4	JSON mode plus a Pydantic schema
Negative instructions	P3, P4	"Do not invent categories. If unsure, return needs_review."
Short reasoning field	P2, P4	A one-sentence reason per answer
Task decomposition	P1 to P4	One column per call instead of one giant prompt
Self-check (critic)	P4	A second call reviews the first answer for bias
Low temperature	All	temperature = 0 for repeatable output
Iterative refinement	All	Versions v1, v2, v3, each scored
Spec-first, test-first coding	B1 to B8	Ask for tests together with the code

Context block placed at the start of every build prompt:

I am building a Streamlit tool in Python 3.11 that cleans and validates CSV files. Stack: pandas, Pandera, RapidFuzz, google-genai, Streamlit, Jinja2, pytest, python-dotenv, python-dateutil, phonenumbers, pydantic. Create only the project skeleton, with no logic yet:

- folders: cleaner/, cleaner/detectors/, templates/, tests/, outputs/, prompt_log/runs/
- empty **init**.py files and empty module files named: loader.py, profiler.py, llm.py, prompts.py, rules.py, schema.py, report.py, and in detectors/: missing.py, duplicates.py, formats.py, invalid.py
- requirements.txt with these libraries (no version pins), README.md with a title and a one-line description
- do not touch the data/ folder or any .csv/.json files, and do not read .env Show me the file tree when done.

5.2 Build prompts (B1 to B8)

For each build prompt, paste a screenshot of the prompt you sent and a screenshot of the output you received. Then note what you had to fix by hand.

B1: Loader and profiler

Files built	loader.py, profiler.py
Core ask	Read the CSV as text; convert blanks and fake nulls (N/A, NA, null, nan, none, -, ?, Unknown) to null; profile each column (null %, unique count, top values, samples); include tests.
AI tool used	Antigravity
Attempts needed	1
Fixed by hand	Nothing
Git commit	40c96736455d62470133fcaa292c66fbbaa5f35c

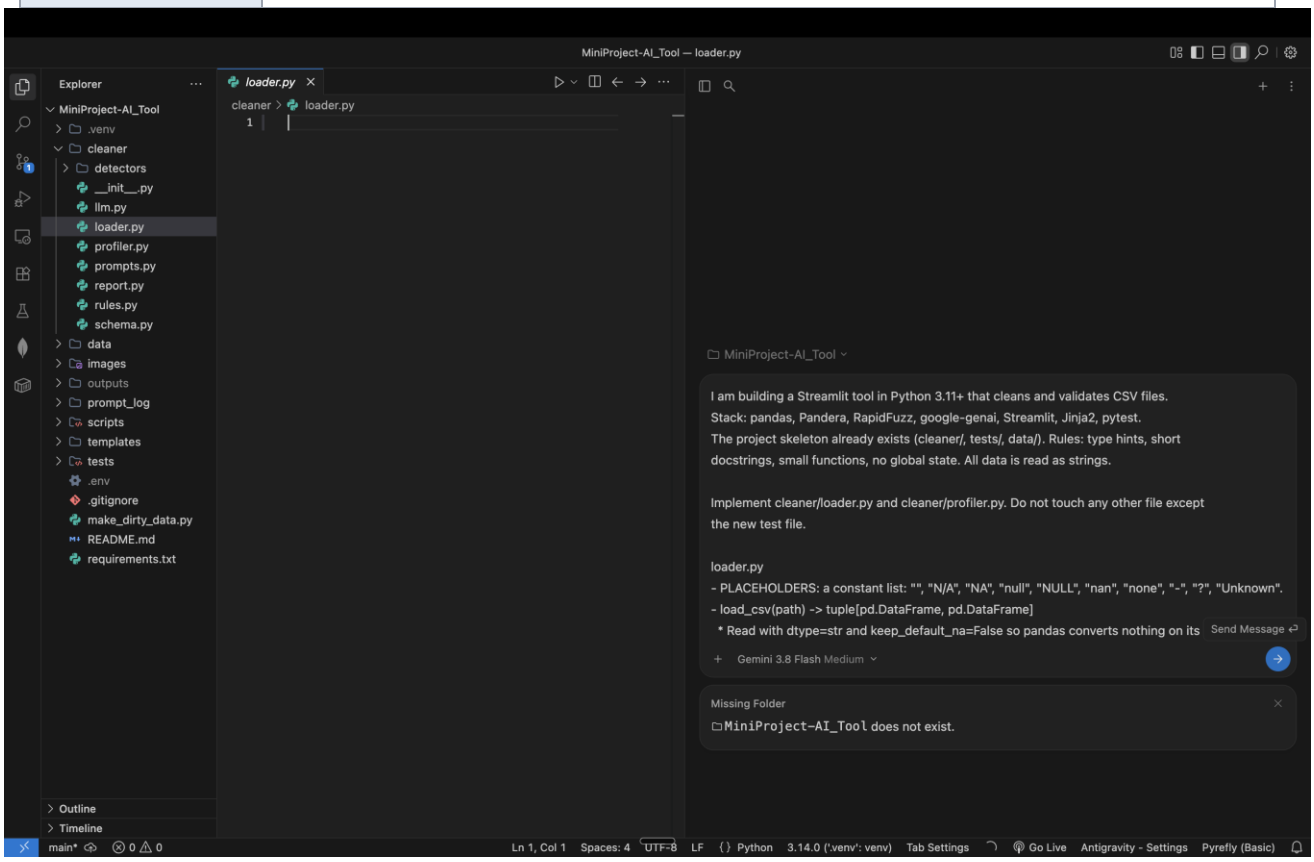


Figure 5: B1: the prompt sent to the AI assistant

B1 prompt text (transcribed from Figure 5)

```
I am building a Streamlit tool in Python 3.11+ that cleans and validates CSV files.
Stack: pandas, Pandera, RapidFuzz, google-genai, Streamlit, Jinja2, pytest.
The project skeleton already exists (cleaner/, tests/, data/). Rules: type hints, short docstrings, small functions,
no global state. All data is read as strings.

Implement cleaner/loader.py and cleaner/profiler.py. Do not touch any other file except the new test file.

loader.py
- PLACEHOLDERS: a constant list: "", "N/A", "NA", "null", "NULL", "nan", "none", "-", "?", "Unknown".
- load_csv(path) -> tuple[pd.DataFrame, pd.DataFrame]
```

```
* Read with dtype=str and keep_default_na=False so pandas converts nothing on its own.
* A cell is "blank" if it is empty after stripping, and a "placeholder" if its stripped, lowercase value matches
an entry in PLACEHOLDERS. Replace both with pd.NA.
* Return (cleaned_df, null_log). null_log has the columns [row, column, original_value, kind], where row is the 1-
based data row (the header is not counted) and kind is "blank" or "placeholder".
```

```
profiler.py
- profile(df) -> pd.DataFrame with one row per column: column, null_count, null_pct (2 decimals), unique_count,
top_values (the 5 most common non-null values with their counts), samples (up to 5 non-null values).

Tests (tests/test_loader_profiler.py)
- placeholders and blanks become null and are logged with the right kind;
- null_pct and unique_count are correct on a small example;
- data/orders_dirty.csv loads with 545 rows and 16 columns.
Type hints, short docstrings, no global state. Show me the command to run the tests.
```

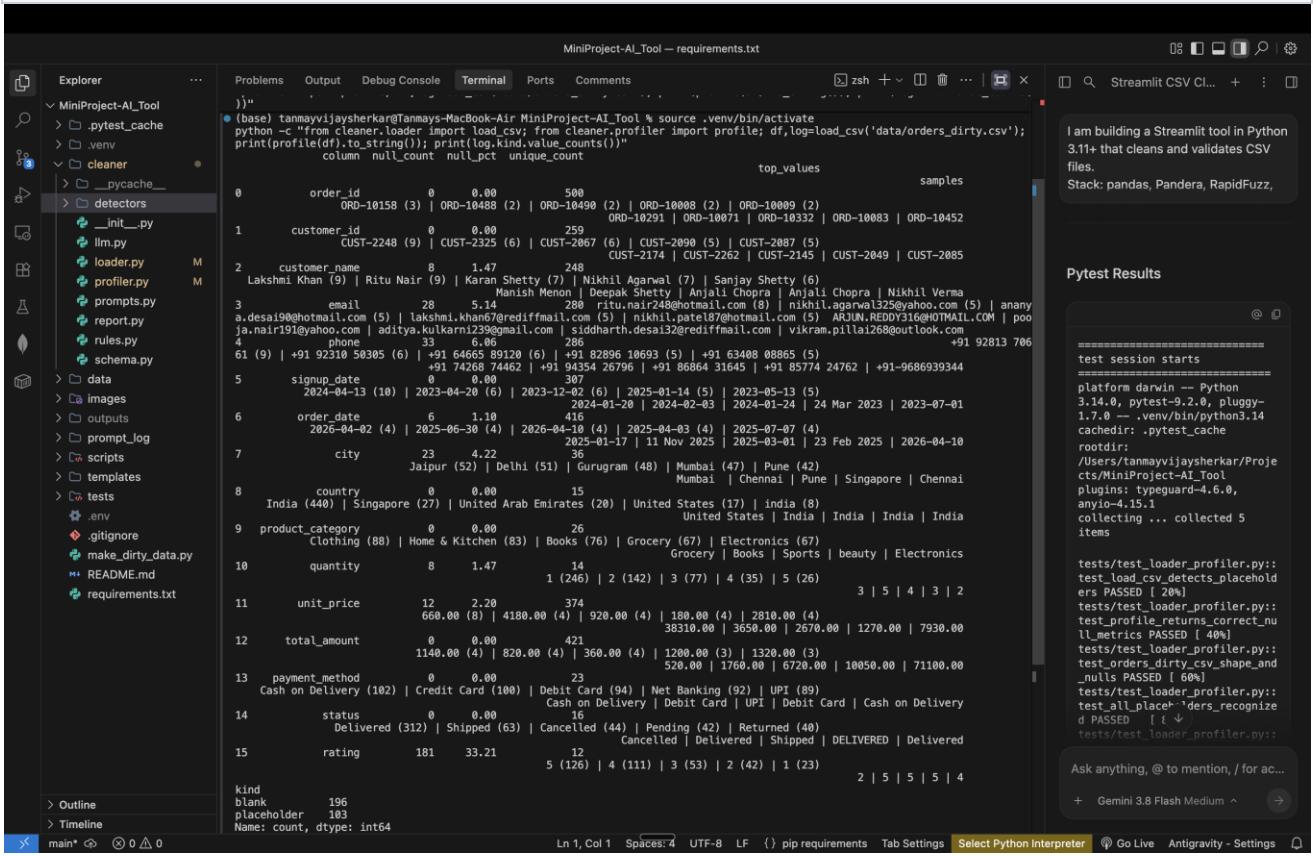


Figure 6: *B1: the AI output (code or reply)*

B2: Detectors

Files built	detectors/missing.py, duplicates.py, formats.py, invalid.py
Core ask	Four detector modules for missing values, exact and fuzzy duplicates, format inconsistencies and invalid values; no LLM calls; one test per check; state the day-first date assumption.
AI tool used	Antigravity
Attempts needed	1
Fixed by hand	Added Introduction to what we are building (context)
Git commit	10a23b34250d05a616e1b85a897bc485e9fdedfe

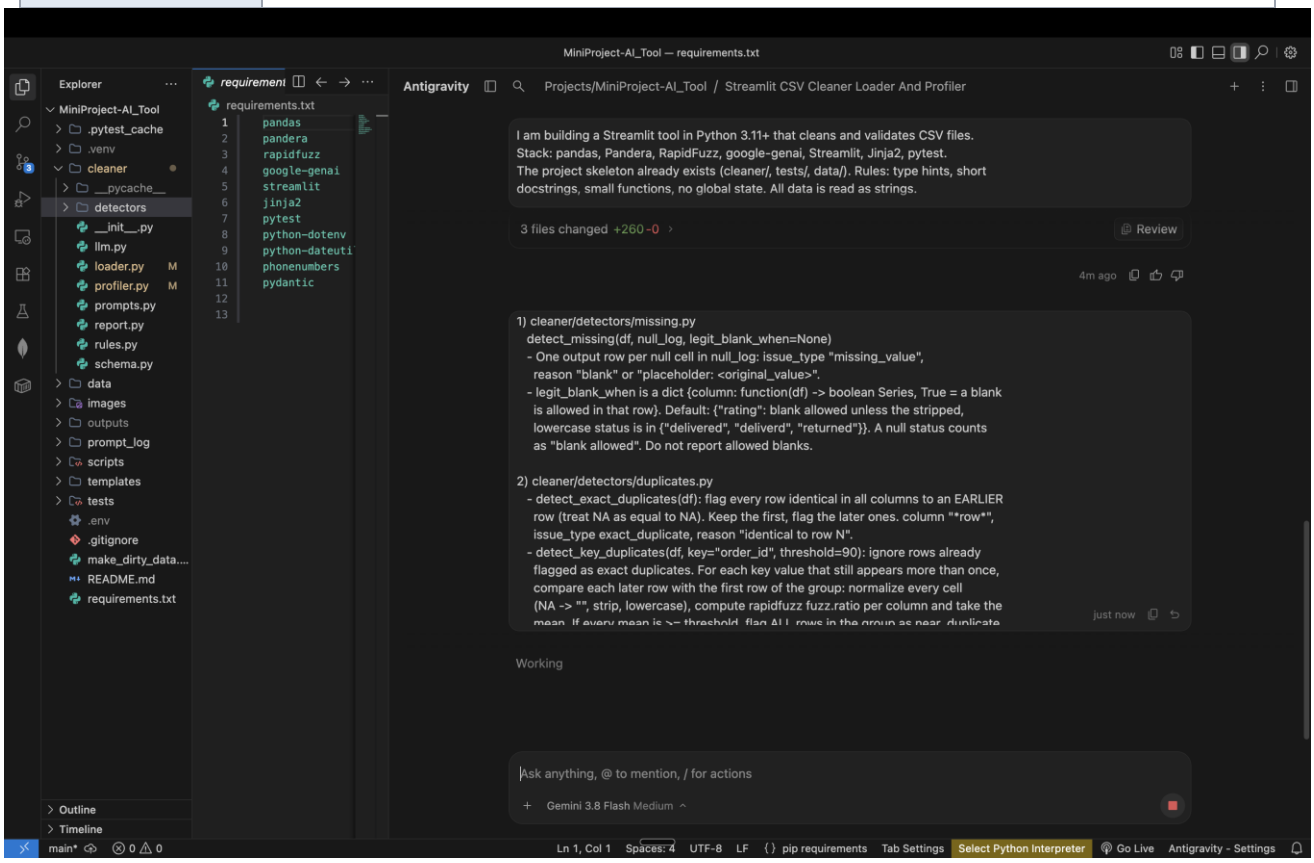


Figure 7: *B2: the prompt sent to the AI assistant*

B2 prompt text (transcribed from Figure 7)

```
I am building a Streamlit tool in Python 3.11+ that cleans and validates CSV files.
Stack: pandas, Pandera, RapidFuzz, google-genai, Streamlit, Jinja2, pytest.
The project skeleton already exists (cleaner/, tests/, data/). Rules: type hints, short docstrings, small functions,
no global state. All data is read as strings.

1) cleaner/detectors/missing.py
detect_missing(df, null_log, legit_blank_when=None)
- One output row per null cell in null_log: issue_type "missing_value", reason "blank" or "placeholder:
<original_value>".
- legit_blank_when is a dict {column: function(df) -> boolean Series, True = a blank is allowed in that row}.
Default: {"rating": blank allowed unless the stripped, lowercase status is in {"delivered", "deliverd",
"returned"}}. A null status counts as "blank allowed". Do not report allowed blanks.

2) cleaner/detectors/duplicates.py
- detect_exact_duplicates(df): flag every row identical in all columns to an EARLIER row (treat NA as equal to NA).
Keep the first, flag the later ones. column "*row*", issue_type exact_duplicate, reason "identical to row N".
- detect_key_duplicates(df, key="order_id", threshold=90): ignore rows already flagged as exact duplicates. For each
key value that still appears more than once, compare each later row with the first row of the group: normalize every
cell (NA -> "", strip, lowercase), compute rapidfuzz fuzz.ratio per column and take the mean. If every mean is >=
threshold, flag ALL rows in the group as near_duplicate (column "*row*", reason "near-duplicate of row N").
Otherwise flag all rows in the group as duplicate_key_conflict, reason "same order_id, different content".

3) cleaner/detectors/formats.py
- detect_formats(df, column_types): for each date column, find the dominant pattern (YYYY-MM-DD) and flag every
value written in another pattern (DD/MM/YYYY, YYYY/MM/DD, "4 Mar 2025" and so on) as format_inconsistency. State
```

```
clearly in the docstring that all slash dates are read as DAY-FIRST (DD/MM).
- Do the same for phone numbers (use the phonenumbers library with the country from the row), money (Rs., INR, rupee
sign, commas), casing and stray spaces, and for category columns (country, city, product_category, payment_method,
status): flag values that differ from the most common spelling only by case, spaces or a known alias.

4) cleaner/detectors/invalid.py
- invalid email (regex); rating outside 1 to 5 or not an integer; quantity that is zero, negative or not a positive
integer; impossible or future dates; signup_date after order_date; total_amount not equal to quantity x unit_price
(within 0.01); price outliers by the IQR rule within each product_category.

Every detector returns a DataFrame with the columns [row, column, value, issue_type, reason] and never changes the
input. No LLM calls anywhere. Add one pytest per check in tests/test_detectors_a.py. Type hints, short docstrings,
no global state.
```

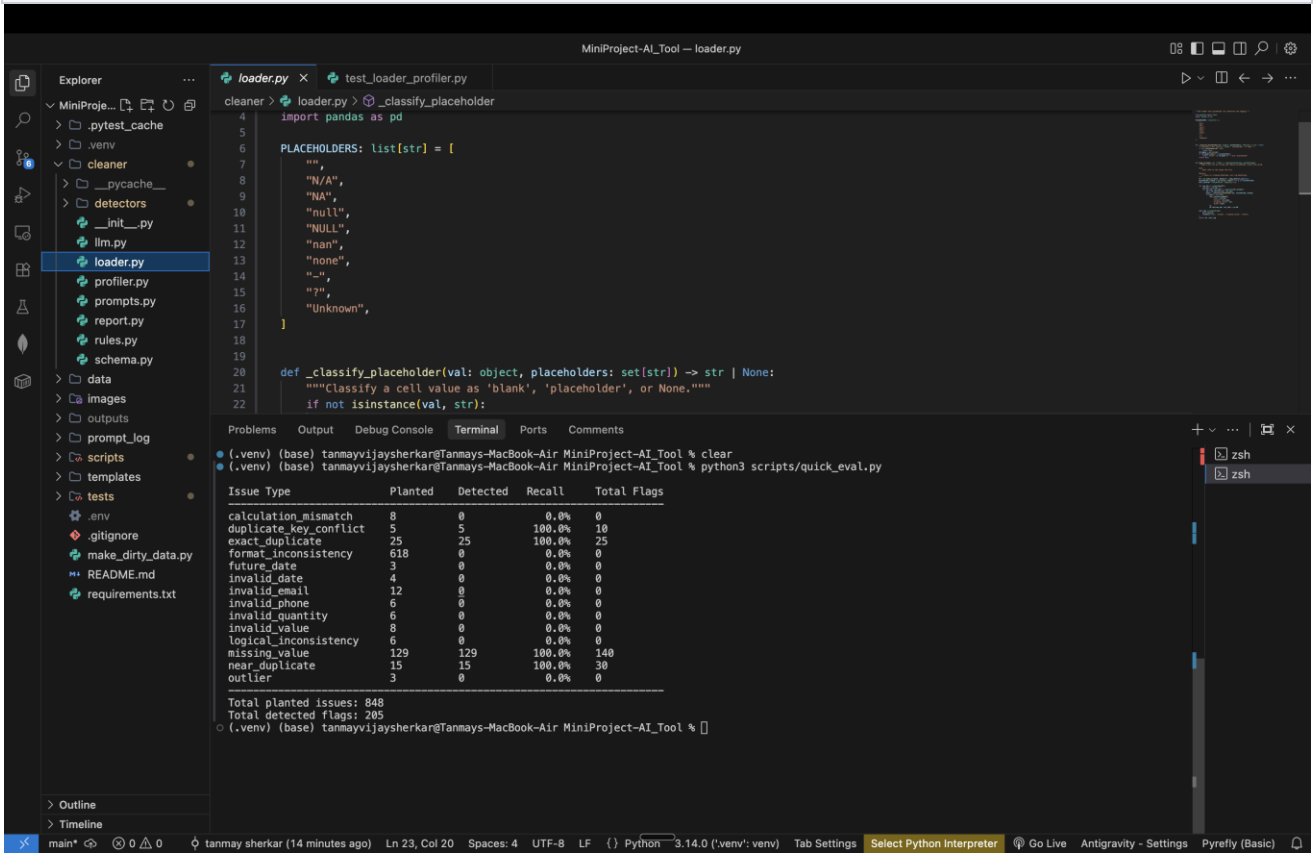


Figure 8: *B2: the AI output (code or reply)*

B3: Gemini client

Files built	llm.py
Core ask	call_gemini(prompt, schema, model, temperature=0) in JSON mode; retry on 429 with backoff; disk cache by prompt hash; save every request and response; fake client for offline tests.
AI tool used	Google AI Studios
Attempts needed	2
Fixed by hand	Gave Error of model not accepted but the issue was that the google account was a organization account
Git commit	069338f32510ffd6e890a8d242d2c00a12ae603d

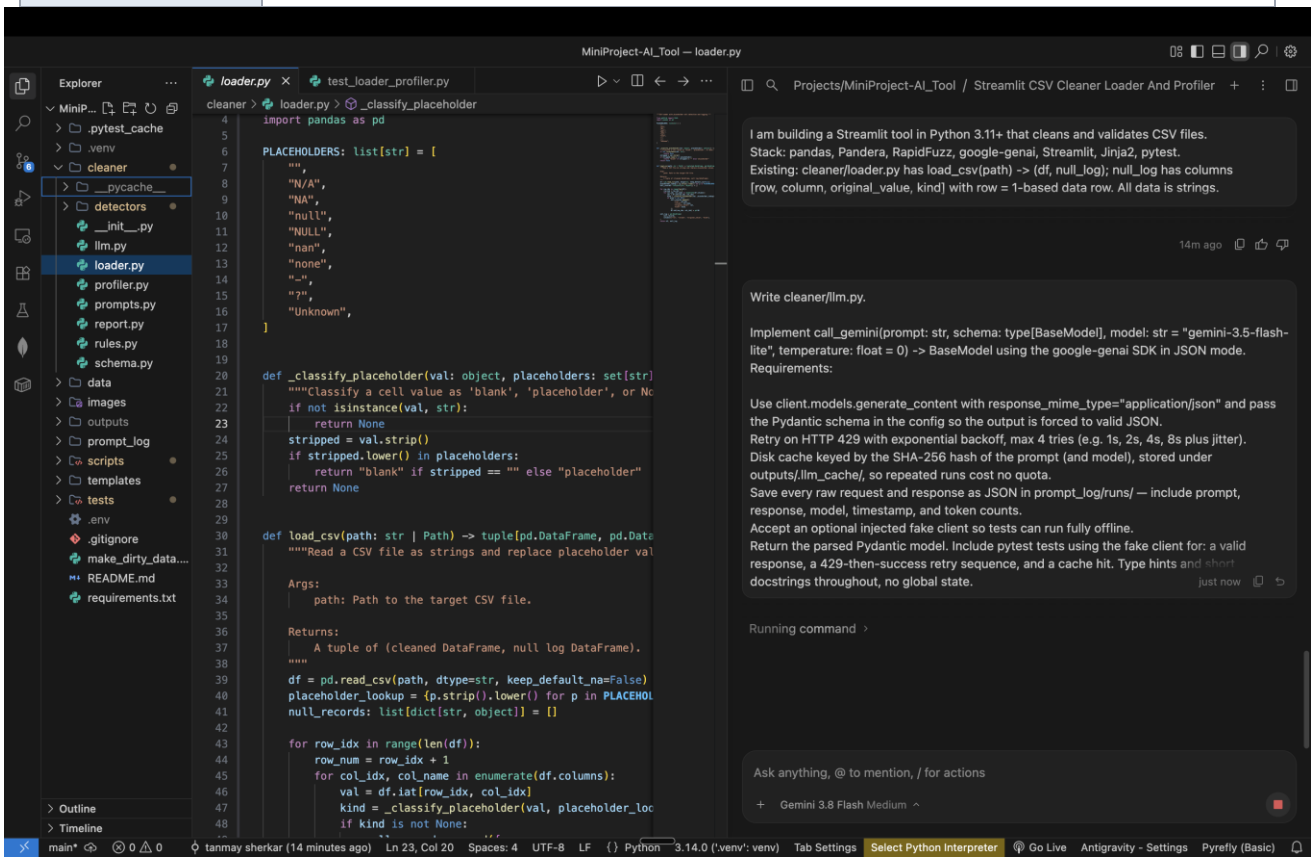


Figure 9: B3: the prompt sent to the AI assistant

B3 prompt text (transcribed from Figure 9)

```
I am building a Streamlit tool in Python 3.11+ that cleans and validates CSV files.
Stack: pandas, Pandera, RapidFuzz, google-genai, Streamlit, Jinja2, pytest.
Existing: cleaner/loader.py has load_csv(path) -> (df, null_log); null_log has columns [row, column, original_value,
kind] with row = 1-based data row. All data is strings.

Write cleaner/llm.py.

Implement call_gemini(prompt: str, schema: type[BaseModel], model: str = "gemini-3.5-flash-lite", temperature: float
= 0) -> BaseModel using the google-genai SDK in JSON mode. Requirements:

Use client.models.generate_content with response_mime_type="application/json" and pass the Pydantic schema in the
config so the output is forced to valid JSON.
Retry on HTTP 429 with exponential backoff, max 4 tries (e.g. 1s, 2s, 4s, 8s plus jitter).
Disk cache keyed by the SHA-256 hash of the prompt (and model), stored under outputs/.llm_cache/, so repeated runs
cost no quota.
Save every raw request and response as JSON in prompt_log/runs/ - include prompt, response, model, timestamp, and
token counts.
Accept an optional injected fake client so tests can run fully offline.
Return the parsed Pydantic model. Include pytest tests using the fake client for: a valid response, a 429-then-
success retry sequence, and a cache hit. Type hints and short docstrings throughout, no global state.
```

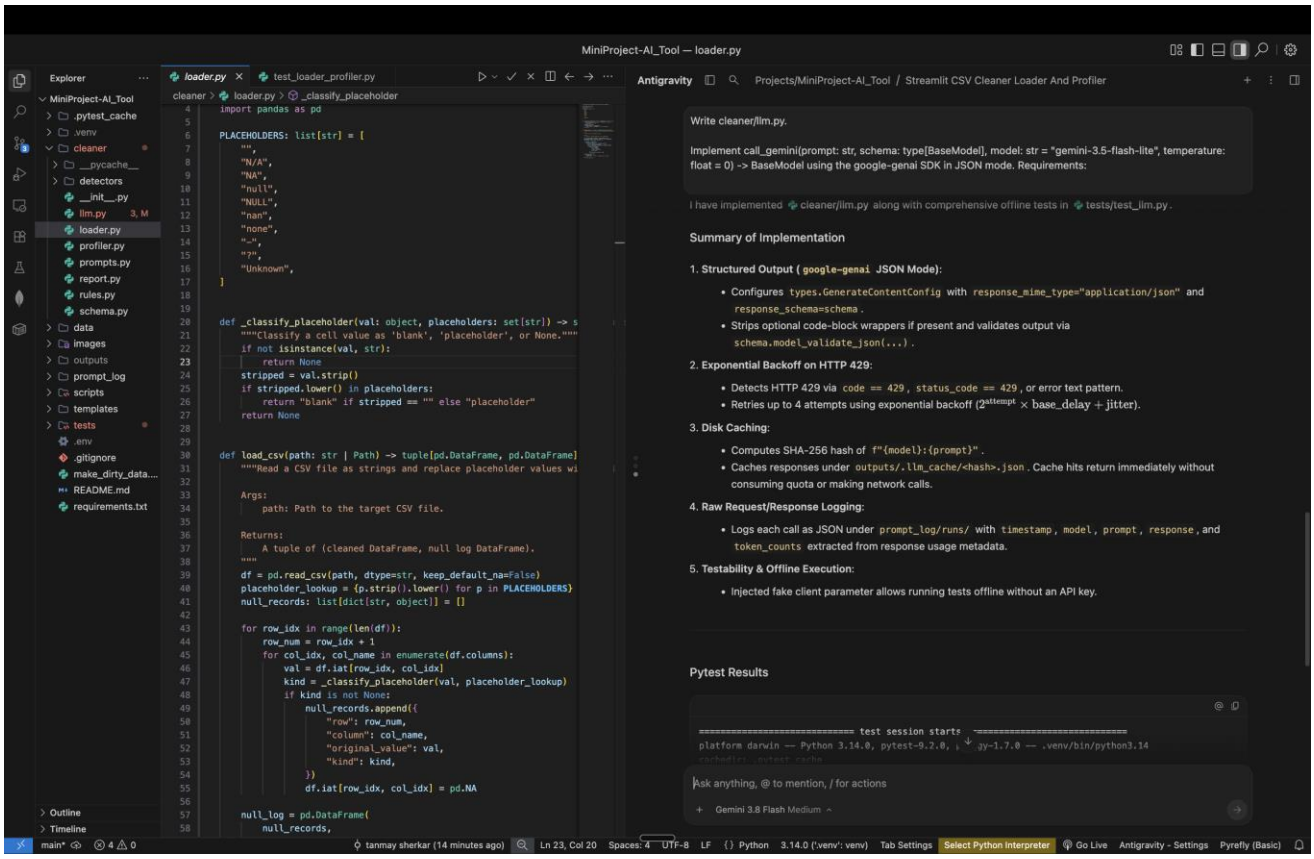


Figure 10: *B3: the AI output (code or reply)*

B4: Prompt debugging (meta-prompt)

Files built	prompts.py
Core ask	Used again and again: "Here is my prompt, here is the output it gave, here is what I wanted. Explain why it failed, then give a new version that changes as little as possible."
AI tool used	Antigravity
Attempts needed	1
Fixed by hand	Nothing
Git commit	-

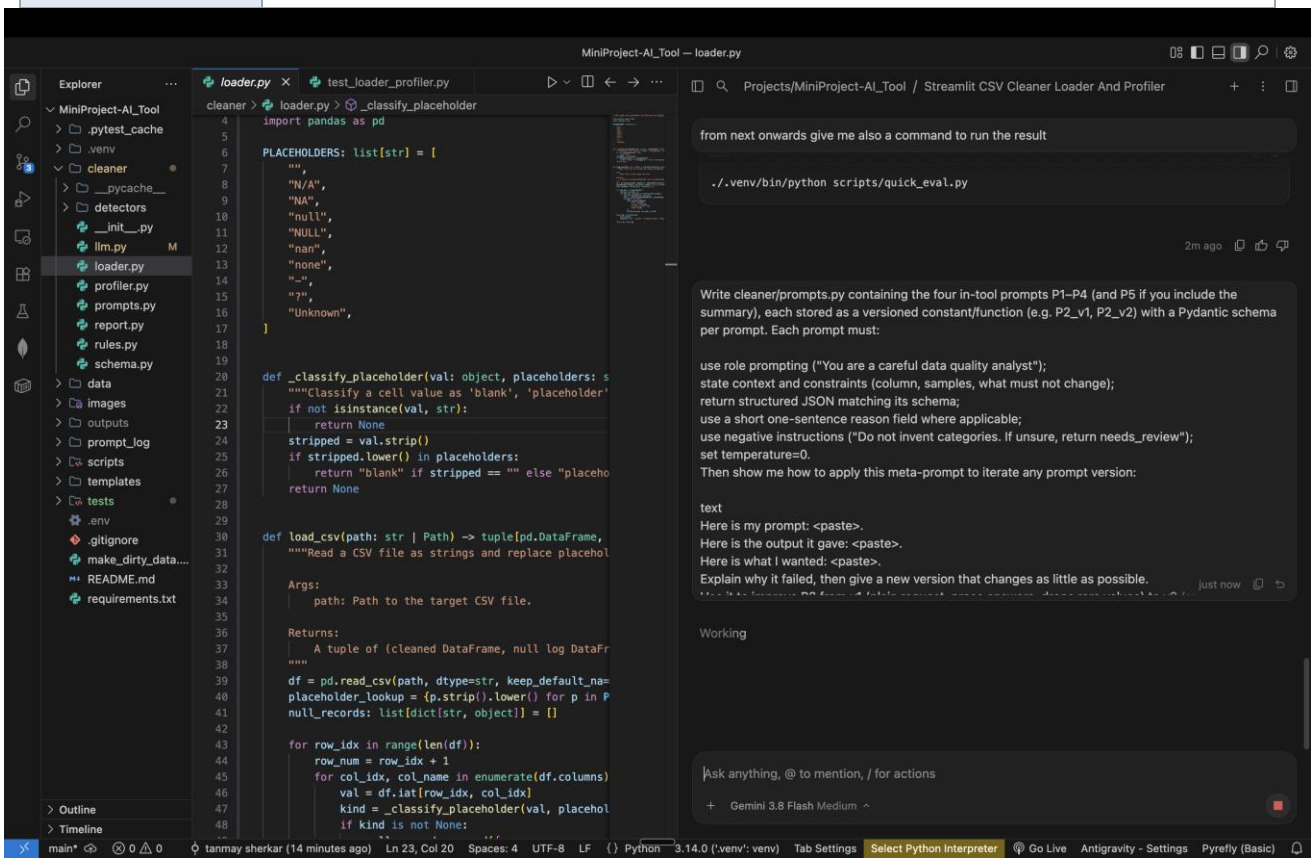


Figure 11: B4: the prompt sent to the AI assistant

B4 prompt text (transcribed from Figure 11)

```
Write cleaner/prompts.py containing the four in-tool prompts P1-P4 (and P5 if you include the summary), each stored
as a versioned constant/function (e.g. P2_v1, P2_v2) with a Pydantic schema per prompt. Each prompt must:

use role prompting ("You are a careful data quality analyst");
state context and constraints (column, samples, what must not change);
return structured JSON matching its schema;
use a short one-sentence reason field where applicable;
use negative instructions ("Do not invent categories. If unsure, return needs_review");
set temperature=0.
Then show me how to apply this meta-prompt to iterate any prompt version:

text
Here is my prompt: <paste>.
Here is the output it gave: <paste>.
Here is what I wanted: <paste>.
Explain why it failed, then give a new version that changes as little as possible.
```

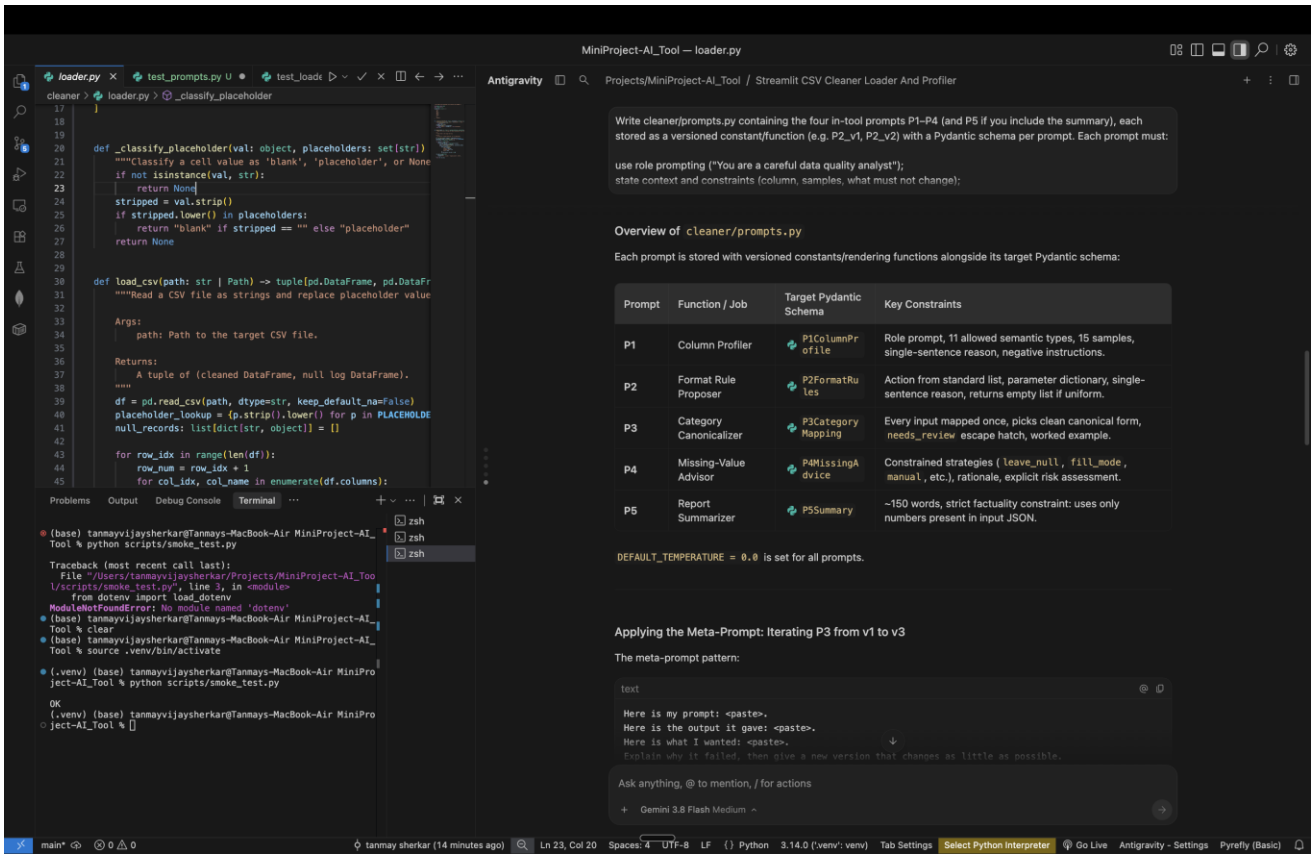


Figure 12: *B4: the AI output (code or reply)*

B5: Rules engine and change log

Files built	rules.py
Core ask	apply_rules(df, rules) returns (clean_df, change_log_df); actions parse_date, strip_trim, title_case, map_values, normalize_phone, parse_money; never mutate the input; tests for each action.
AI tool used	Antigravity
Attempts needed	1
Fixed by hand	Centralized common instructions in Agent.md instead of repeating them in each prompt.
Git commit	1936d218321f3114fdd41663605b6f241d60de42

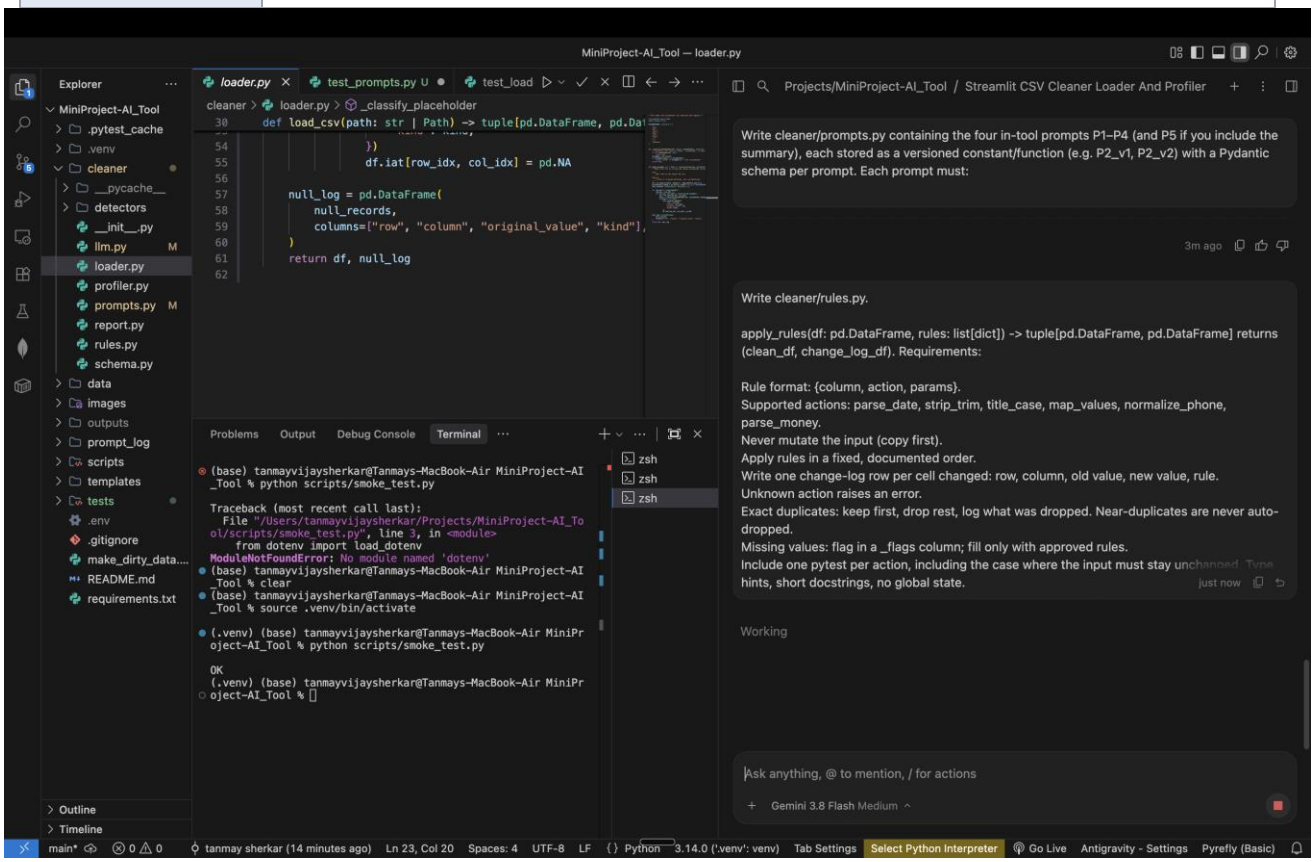


Figure 13: B5: the prompt sent to the AI assistant

B5 prompt text (transcribed from Figure 13)

```
Write cleaner/rules.py.

apply_rules(df: pd.DataFrame, rules: list[dict]) -> tuple[pd.DataFrame, pd.DataFrame] returns (clean_df,
change_log_df). Requirements:

Rule format: {column, action, params}.
Supported actions: parse_date, strip_trim, title_case, map_values, normalize_phone, parse_money.
Never mutate the input (copy first).
Apply rules in a fixed, documented order.
Write one change-log row per cell changed: row, column, old value, new value, rule.
Unknown action raises an error.
Exact duplicates: keep first, drop rest, log what was dropped. Near-duplicates are never auto-dropped.
Missing values: flag in a _flags column; fill only with approved rules.
Include one pytest per action, including the case where the input must stay unchanged. Type hints, short docstrings,
no global state.
```

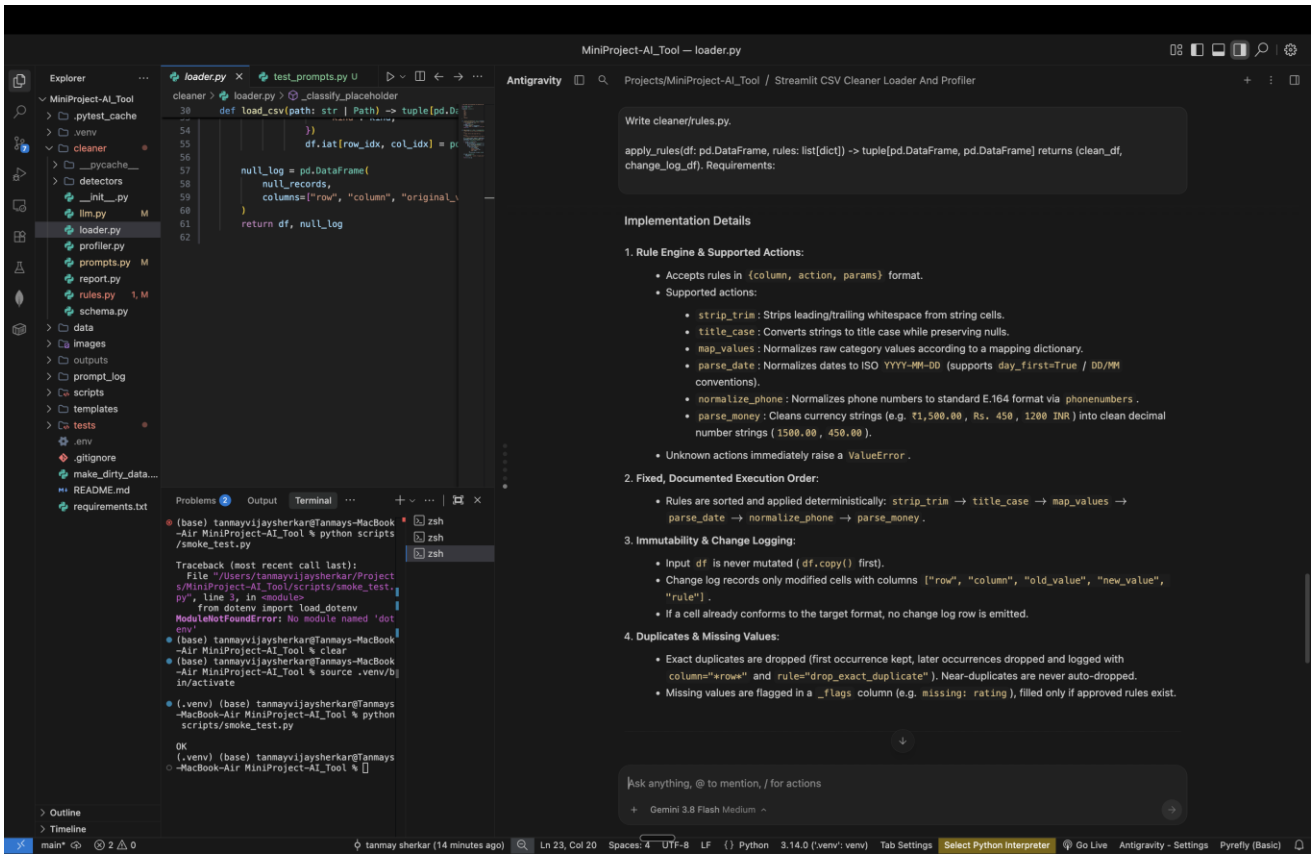


Figure 14: *B5: the AI output (code or reply)*

B6: Pandera validation

Files built	schema.py
Core ask	build_schema(column_types) and validate(df, schema) with lazy=True so all failures are reported at once.
AI tool used	Antigravity
Attempts needed	1
Fixed by hand	My Antigravity tokens were exhausted, so I switched to OpenCode and used Agent.md to preserve workflow state and provide context continuity for the AI.
Git commit	5337f4ccc95ff6fd97eb7ce2f608ee9d81f9fe47

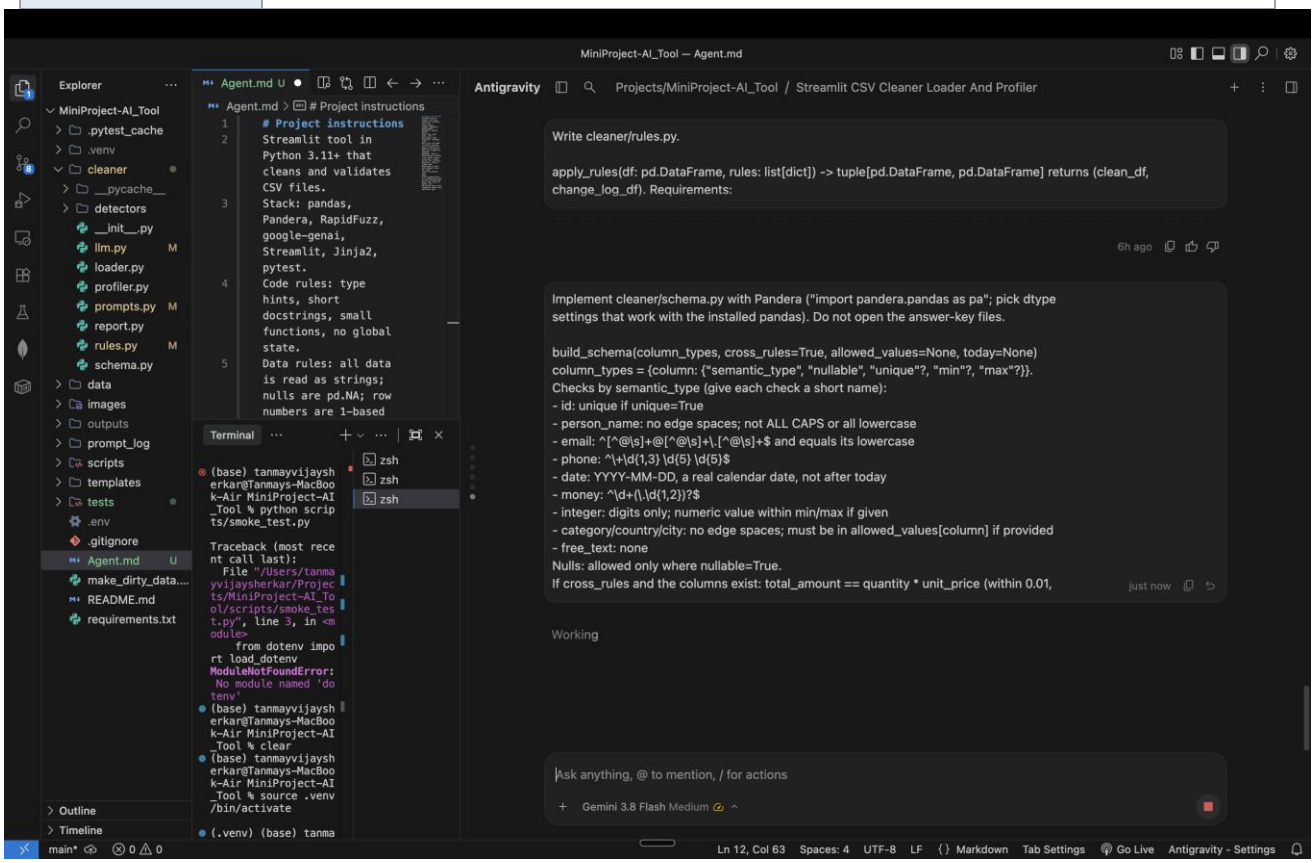


Figure 15: B6: the prompt sent to the AI assistant

B6 prompt text (transcribed from Figure 15)

```
Implement cleaner/schema.py with Pandera ("import pandera.pandas as pa"; pick dtype settings that work with the
installed pandas). Do not open the answer-key files.

build_schema(column_types, cross_rules=True, allowed_values=None, today=None)
column_types = {column: {"semantic_type", "nullable", "unique"?, "min"?, "max"?}}.
Checks by semantic_type (give each check a short name):
- id: unique if unique=True
- person_name: no edge spaces; not ALL CAPS or all lowercase
- email: ^[^@\s]+@[^@\s]+\.[^@\s]+$ and equals its lowercase
- phone: ^\+\d{1,3} \d{5} \d{5}$
- date: YYYY-MM-DD, a real calendar date, not after today
- money: ^\d+(\.\d{1,2})?$
- integer: digits only; numeric value within min/max if given
- category/country/city: no edge spaces; must be in allowed_values[column] if provided
- free_text: none
Nulls: allowed only where nullable=True.
If cross_rules and the columns exist: total_amount == quantity * unit_price (within 0.01, skipping rows where any of
the three is null), and signup_date <= order_date.

validate(df, schema) -> DataFrame [row, column, value, check, reason]: call schema.validate(df, lazy=True) so ALL
failures are reported at once, and convert each failure into one row (row = 1-based data row).
summarize(failures) -> dict with the total failures, counts per column and per check, and a passed flag.

Also create cleaner/orders_config.py with ORDERS_COLUMN_TYPES describing all 16 columns of orders_dirty.csv. Add
```

```
pytest tests, type hints, short docstrings, no global state.
```

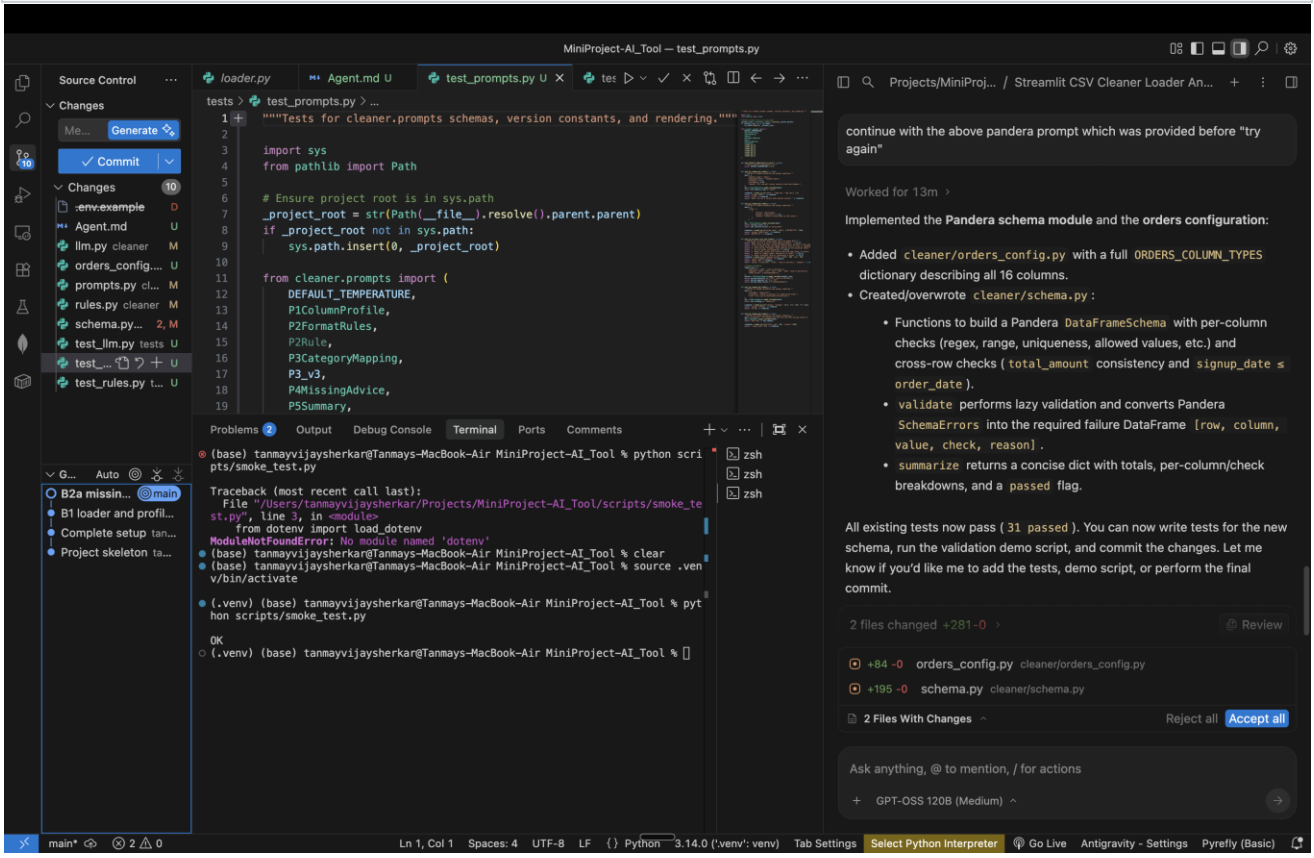


Figure 16: *B6: the AI output (code or reply)*

B7: HTML validation report

Files built	report.py, templates/report.html.j2
Core ask	Self-contained Jinja2 report with inline CSS and the sections listed in Section 6.
AI tool used	Opencode
Attempts needed	2
Fixed by hand	My Antigravity tokens were exhausted, so I switched to OpenCode and provided context using the agent.md logs.
Git commit	a42cf80666d81201e07c760e5aa501fcdfa0c4d1

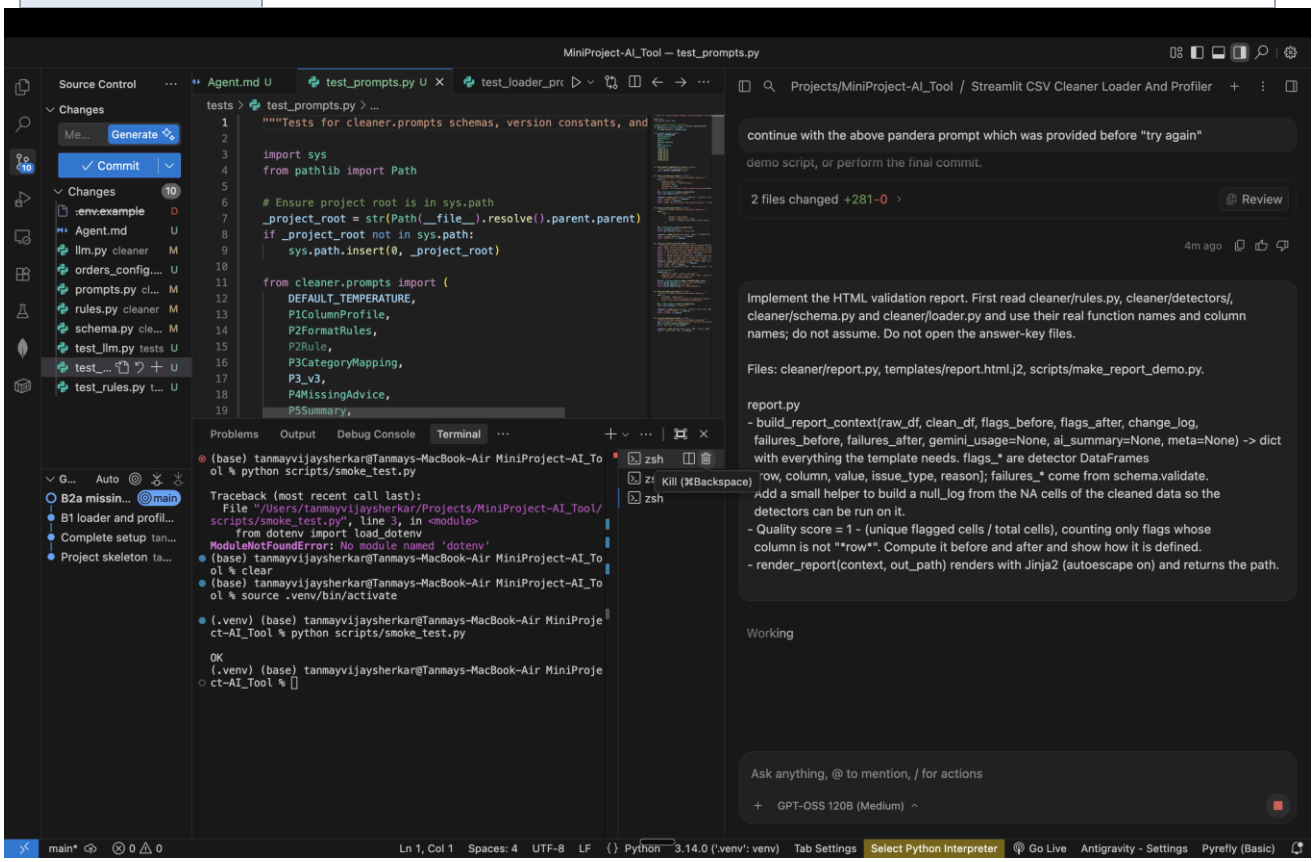


Figure 17: B7: the prompt sent to the AI assistant

B7 prompt text (transcribed from Figure 17)

```
Implement the HTML validation report. First read cleaner/rules.py, cleaner/detectors/, cleaner/schema.py and
cleaner/loader.py and use their real function names and column names; do not assume. Do not open the answer-key
files.

Files: cleaner/report.py, templates/report.html.j2, scripts/make_report_demo.py.

report.py
- build_report_context(raw_df, clean_df, flags_before, flags_after, change_log, failures_before, failures_after,
gemini_usage=None, ai_summary=None, meta=None) -> dict with everything the template needs. flags_* are detector
DataFrames [row, column, value, issue_type, reason]; failures_* come from schema.validate.
```

```
Add a small helper to build a null_log from the NA cells of the cleaned data so the detectors can be run on it.
```

```
- Quality score = 1 - (unique flagged cells / total cells), counting only flags whose column is not "*row*". Compute
it before and after and show how it is defined.
- render_report(context, out_path) renders with Jinja2 (autoescape on) and returns the path.
```

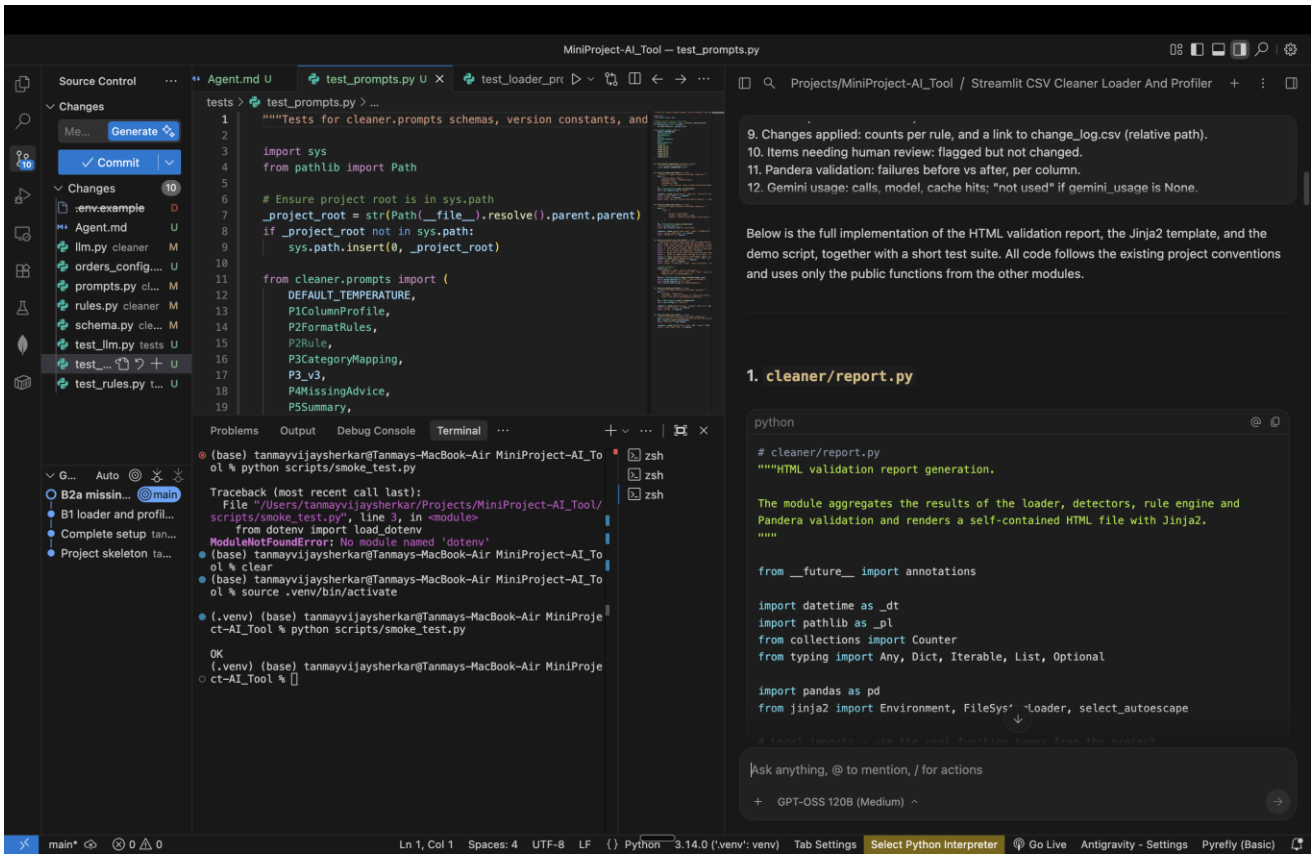


Figure 18: *B7: the AI output (code or reply)*

B8: Streamlit app

Files built	app.py
Core ask	Upload, profile, issues, rule review (accept or reject), before and after preview, downloads; session state; spinner during Gemini calls; clear message if the API key is missing.
AI tool used	Opencode
Attempts needed	1
Fixed by hand	Just signed in to Streamlit Cloud so the project can be deployed.
Git commit	b2c8f526d6e0d6dda4e151ba4b7e5f6c3b270556

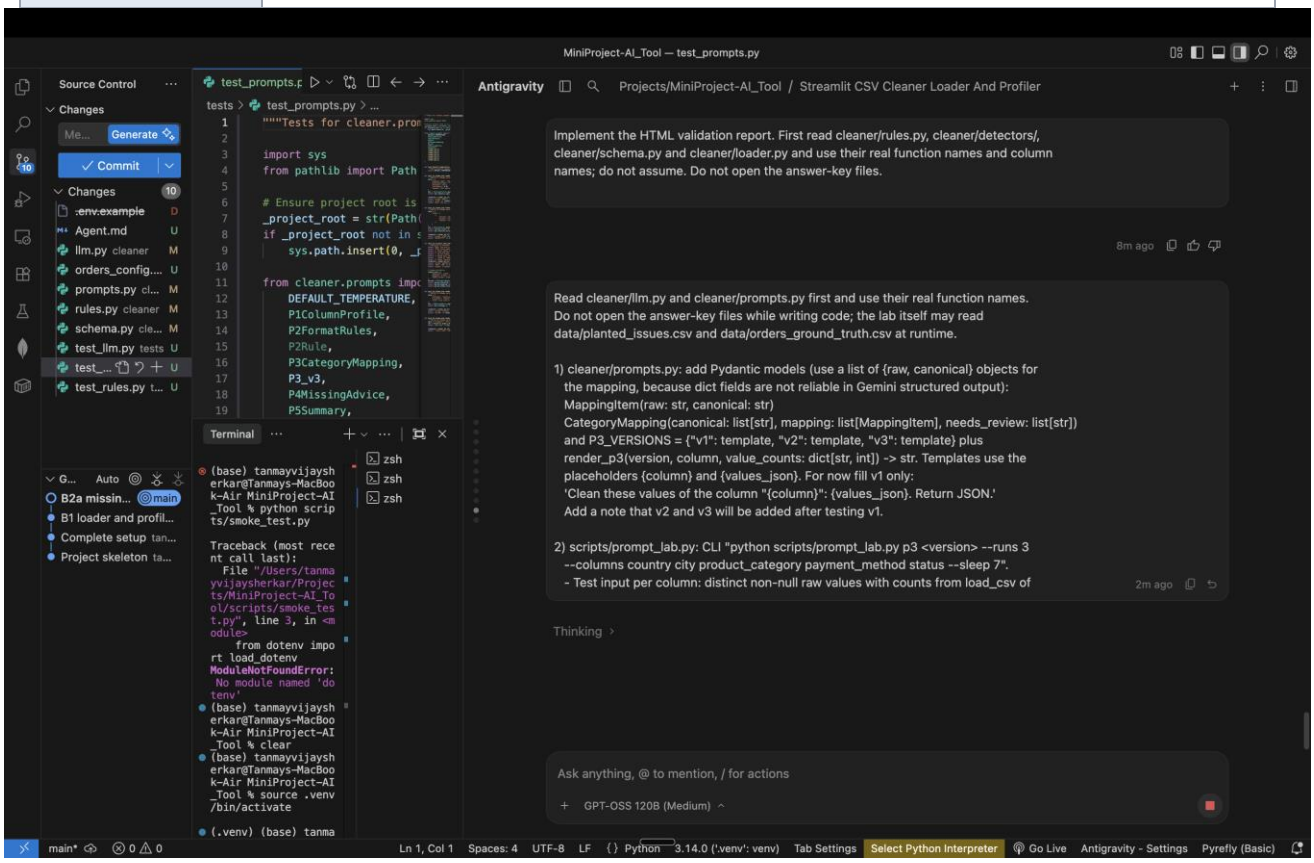


Figure 19: *B8: the prompt sent to the AI assistant*

B8 prompt text (transcribed from Figure 19)

```
I am building a Streamlit tool in Python 3.11+ that cleans and validates CSV files.
Stack: pandas, Pandera, RapidFuzz, google-genai, Streamlit, Jinja2, pytest.
All modules already exist: cleaner/loader.py, profiler.py, detectors/, llm.py, prompts.py, rules.py, schema.py,
report.py. Read them first and use their real function names; do not assume. Do not open the answer-key files.

Implement app.py (Streamlit). Use a sidebar radio called "Navigation" with six pages and a "Use Gemini" checkbox:
1. Upload CSV: st.file_uploader; load with load_csv; show "Loaded N rows x M columns".
2. Profile: the column profile table; if Use Gemini is on, show the P1 result for each column ("Detected types").
3. Issues found: three tables (Missing, Duplicates, Formats/Invalid) from the detectors; if Use Gemini is on, show
P4 advice for columns with missing values.
4. Proposed rules: show the P2 and P3 proposals from Gemini, each with an accept and reject control; also a small
form to add a rule by hand (column, action, params as JSON); an "Apply accepted rules" button that calls
apply_rules. Nothing is applied until the user accepts it.
5. Before / after: a sample of the data before and after cleaning.
6. Download: buttons for the cleaned CSV, the change log CSV, the HTML validation report and a zip of prompt_log.

Use st.session_state to keep data between pages. Show st.spinner during every Gemini call. If GEMINI_API_KEY is
missing, show a clear message and keep the rule-based features working. Type hints, short docstrings, no global
state.
```

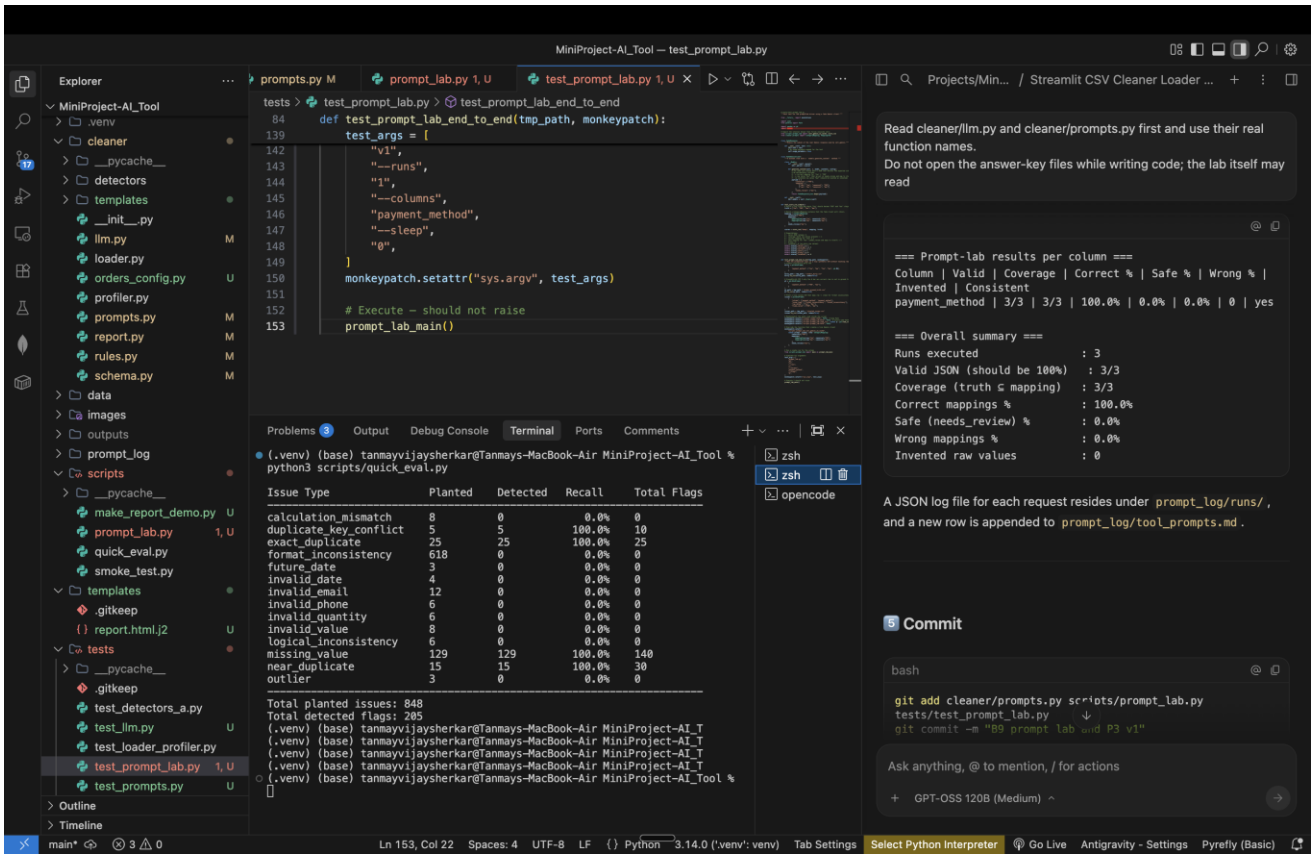


Figure 20: *B8: the AI output (code or reply)*

5.3 In-tool prompts (P1 to P5)

P1 to P5 are the prompts that run inside the application and call Gemini. Each does one narrow job and returns JSON that code validates before anything is used. P3 was built first, together with the prompt lab (B9); P1, P2, P4 and P5 were then added with one build prompt (B10). Each prompt starts as a simple v1 and is only improved after the lab shows a real failure.

ID	Job	Input sent	JSON returned
P1	Column profiler	Column name, 15 sample values, null %	Semantic type, expected format, nullable, confidence
P2	Format rule proposer	Column, type, detected format patterns with counts and examples	List of rules: action, parameters, reason
P3	Category canonicalizer	Column and its distinct values with counts	Canonical list, mapping of every raw value, needs_review list
P4	Missing-value advisor	Column, type, null %, most common values	Strategy, rationale, risk
P5	Report summarizer	Computed statistics as JSON	Plain-English summary (about 150 words, plain text)

How the prompts were tested

scripts/prompt_lab.py runs a prompt version several times on the real dirty data with caching off and temperature 0, and scores each run. Every prompt is scored on valid JSON. P1 is scored on whether its type and nullable answers match a hand-written column configuration; P2 on cleaning accuracy against the ground-truth file after its rules are applied; P3 on correct, safe (needs_review) and wrong mappings, invented values and consistency across runs; P4 on the share of answers inside an allowed-strategy policy; P5 on word count and on numbers that do not appear in the input data.

P3 v3 is the final category prompt. It adds a role, rules that every value must appear once and that unsure values go to needs_review, and one worked example. The prompts are stored in cleaner/prompts.py and every raw request and response is saved under prompt_log/runs.

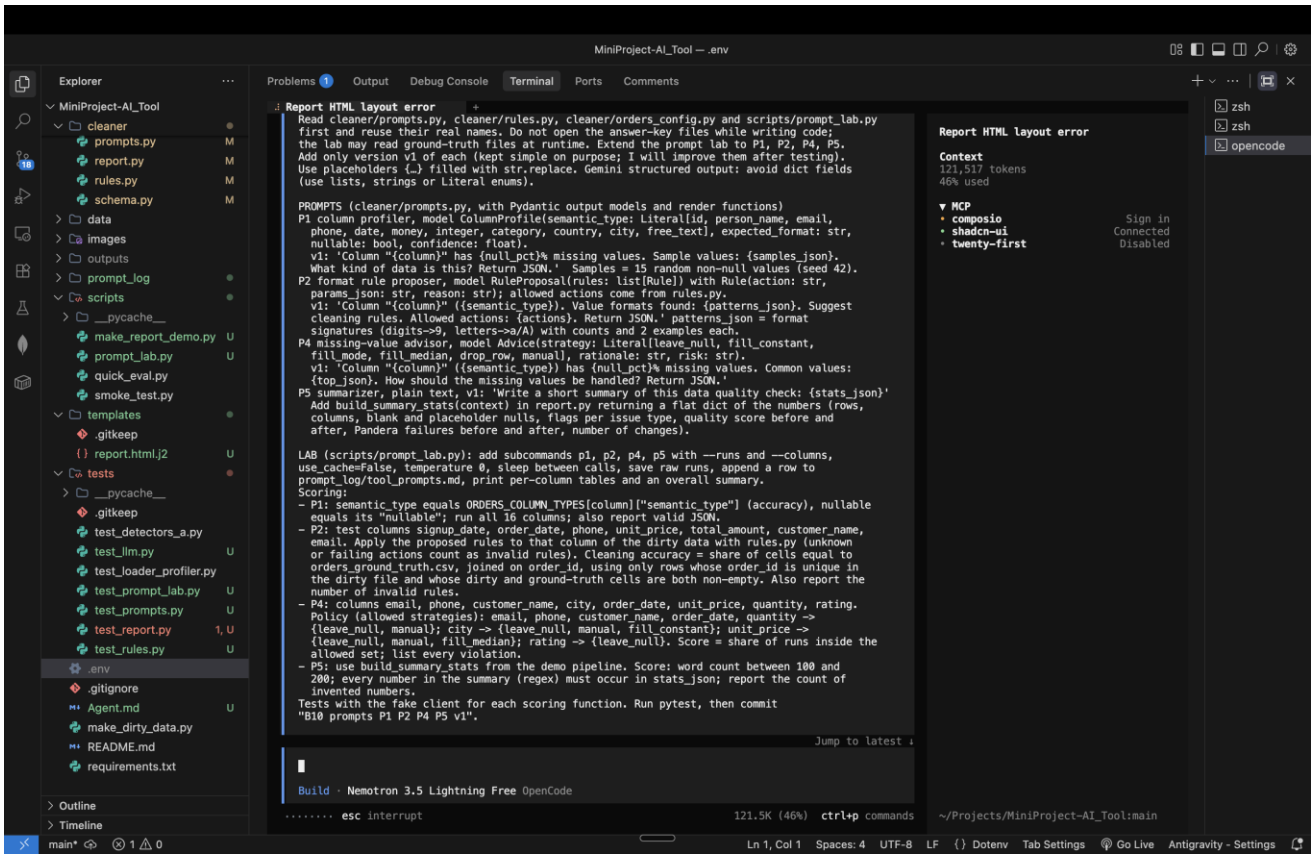


Figure 21: *B10: the single prompt that added P1, P2, P4 and P5*

6. Implementation and Screenshots

6.1 Project structure

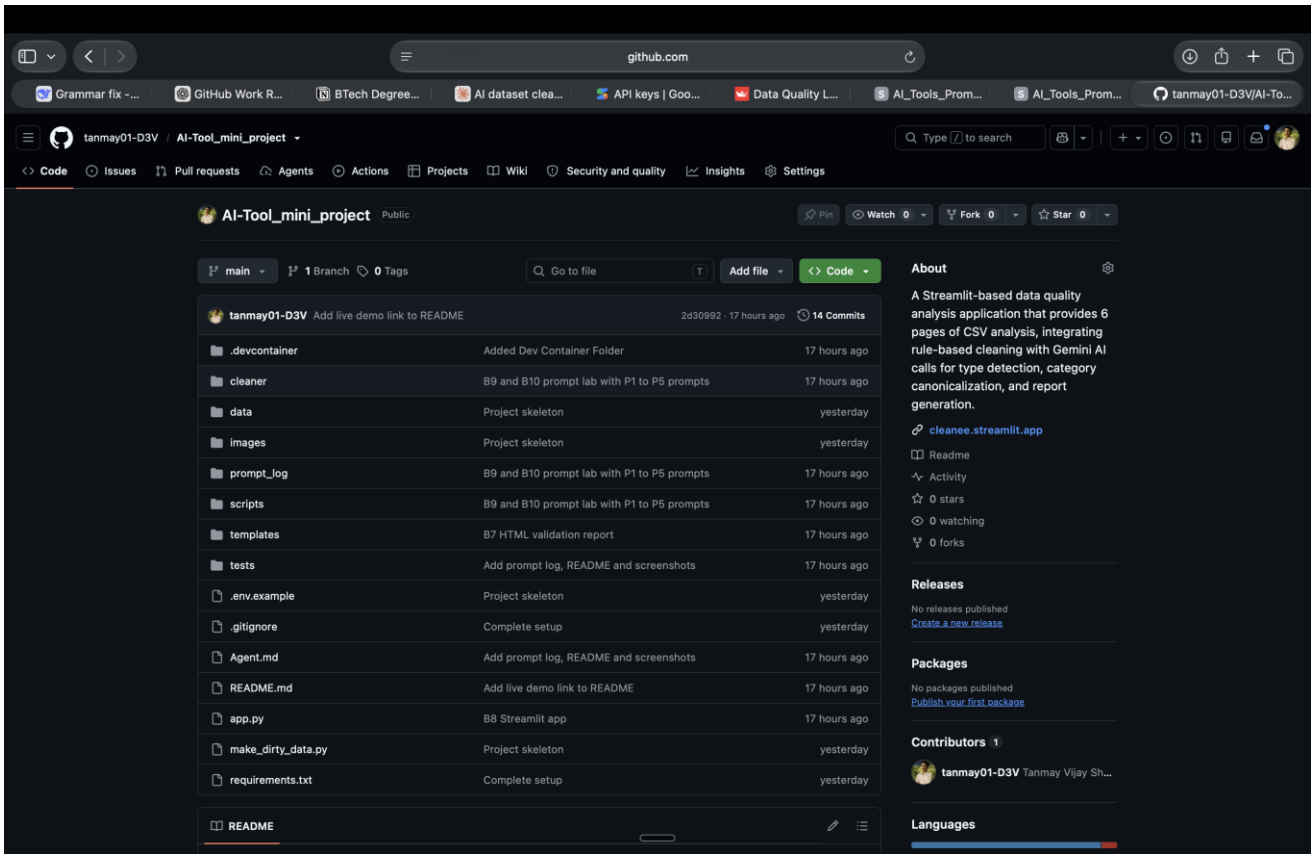


Figure 22: Project folder structure in the code editor or GitHub

6.2 Application screens

The app guides the user through six pages. Add one screenshot for each.

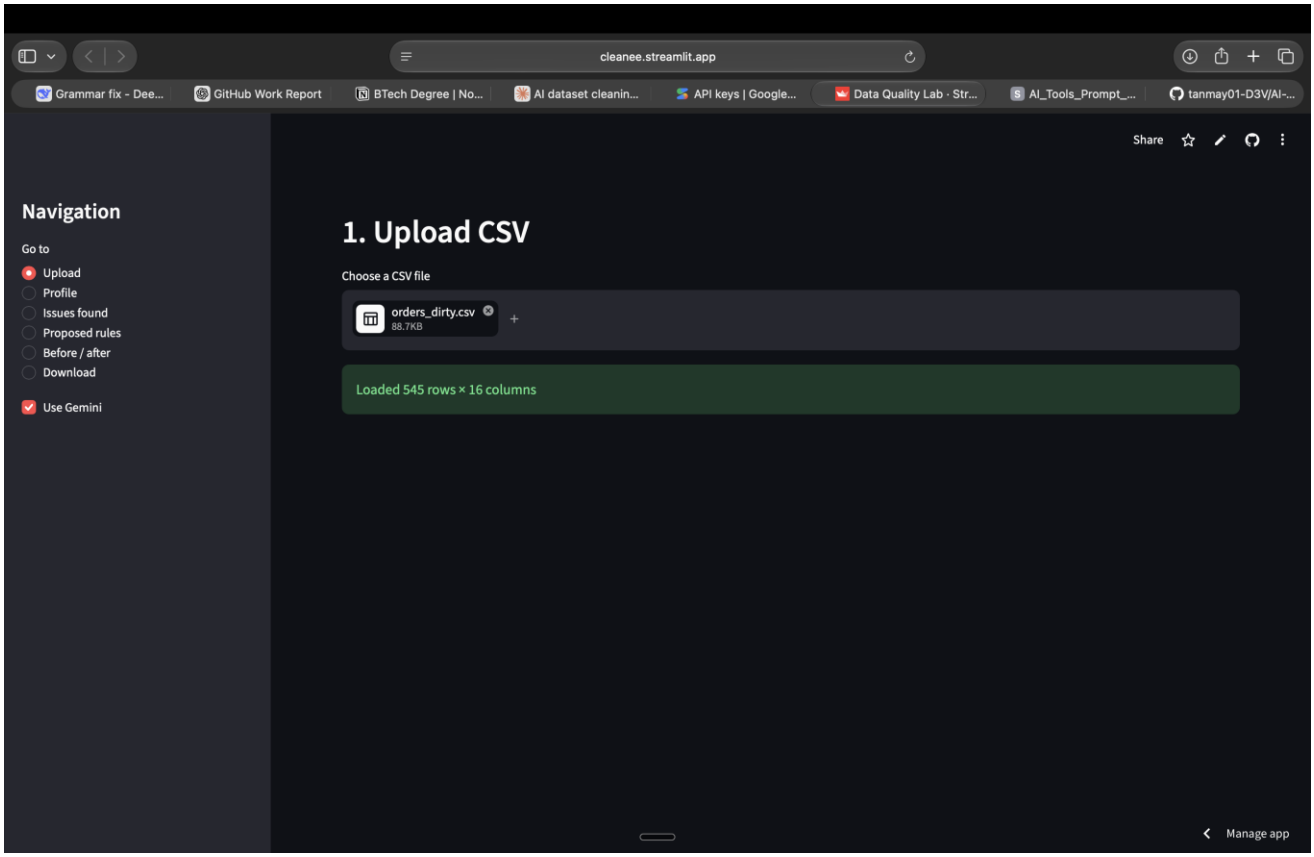


Figure 25: *Upload page: choosing the CSV file*

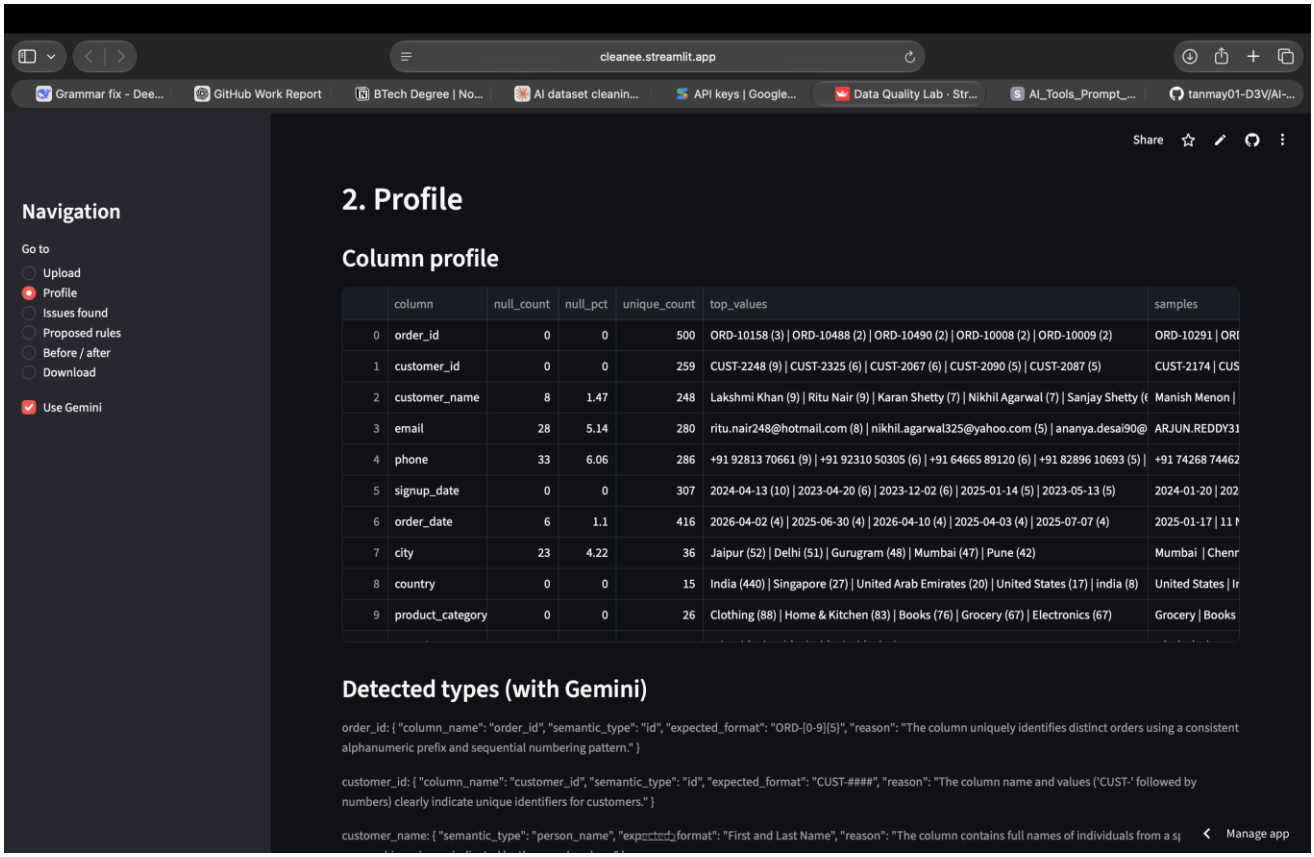


Figure 26: Profile page: per-column statistics

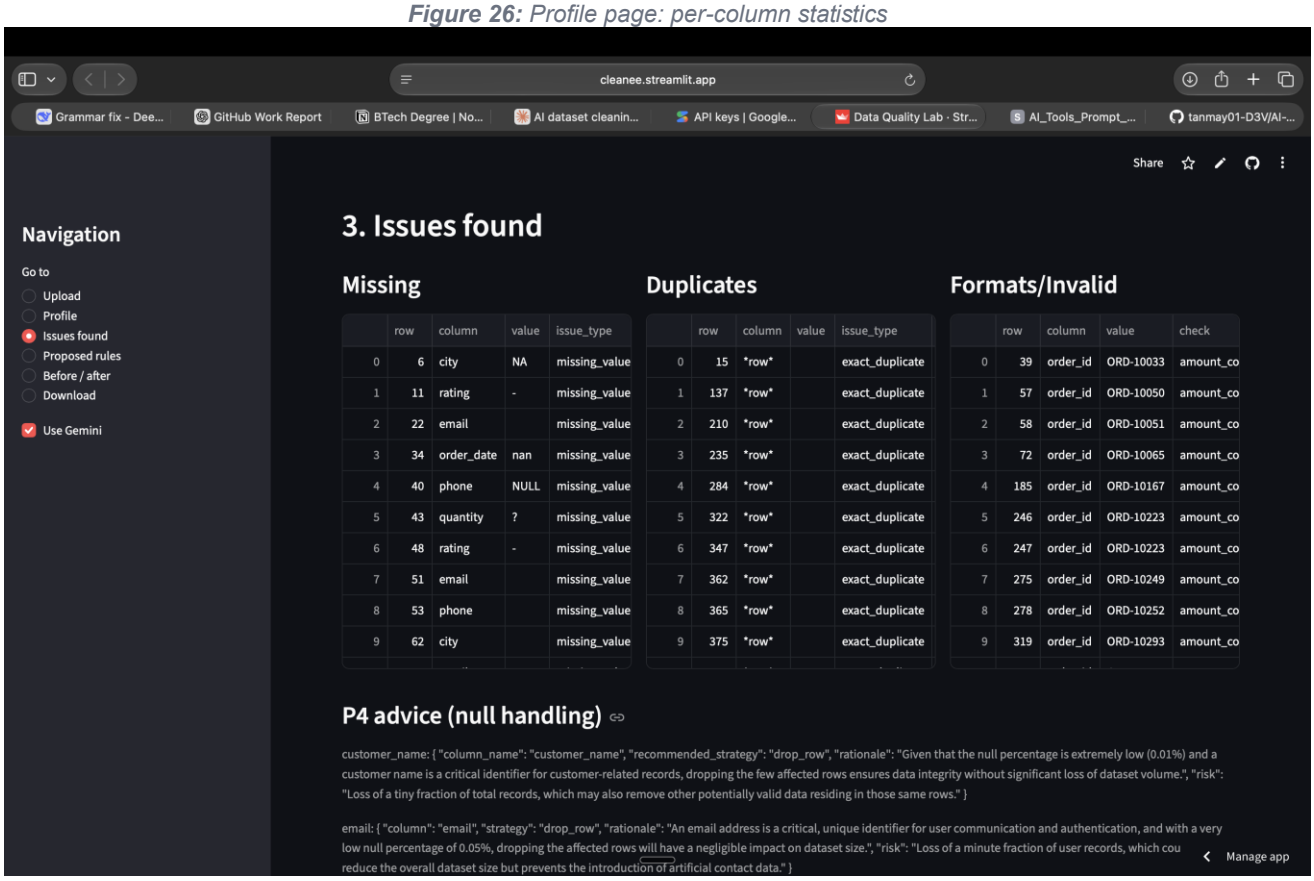


Figure 27: Issues found: missing values, duplicates and format problems

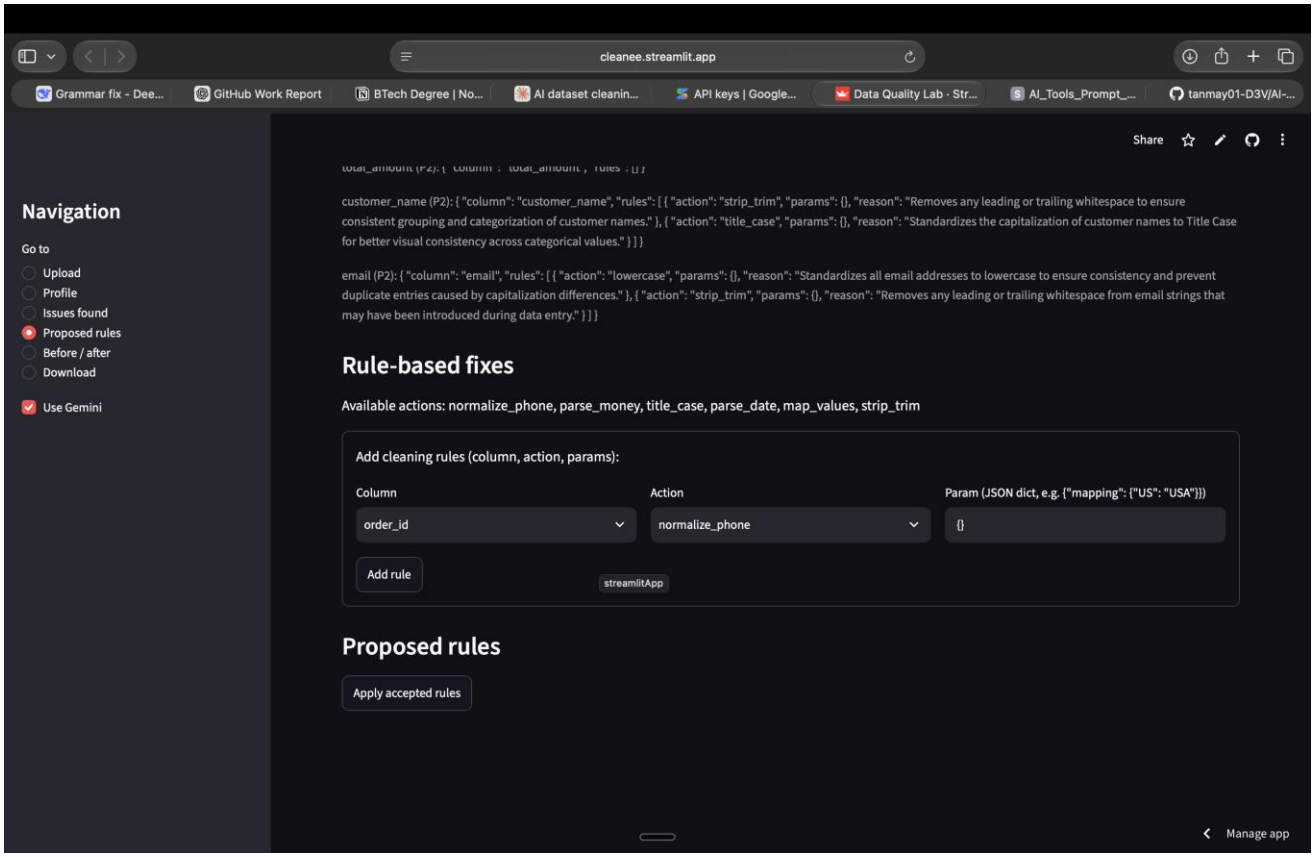


Figure 28: *Proposed rules from Gemini, with accept and reject controls*

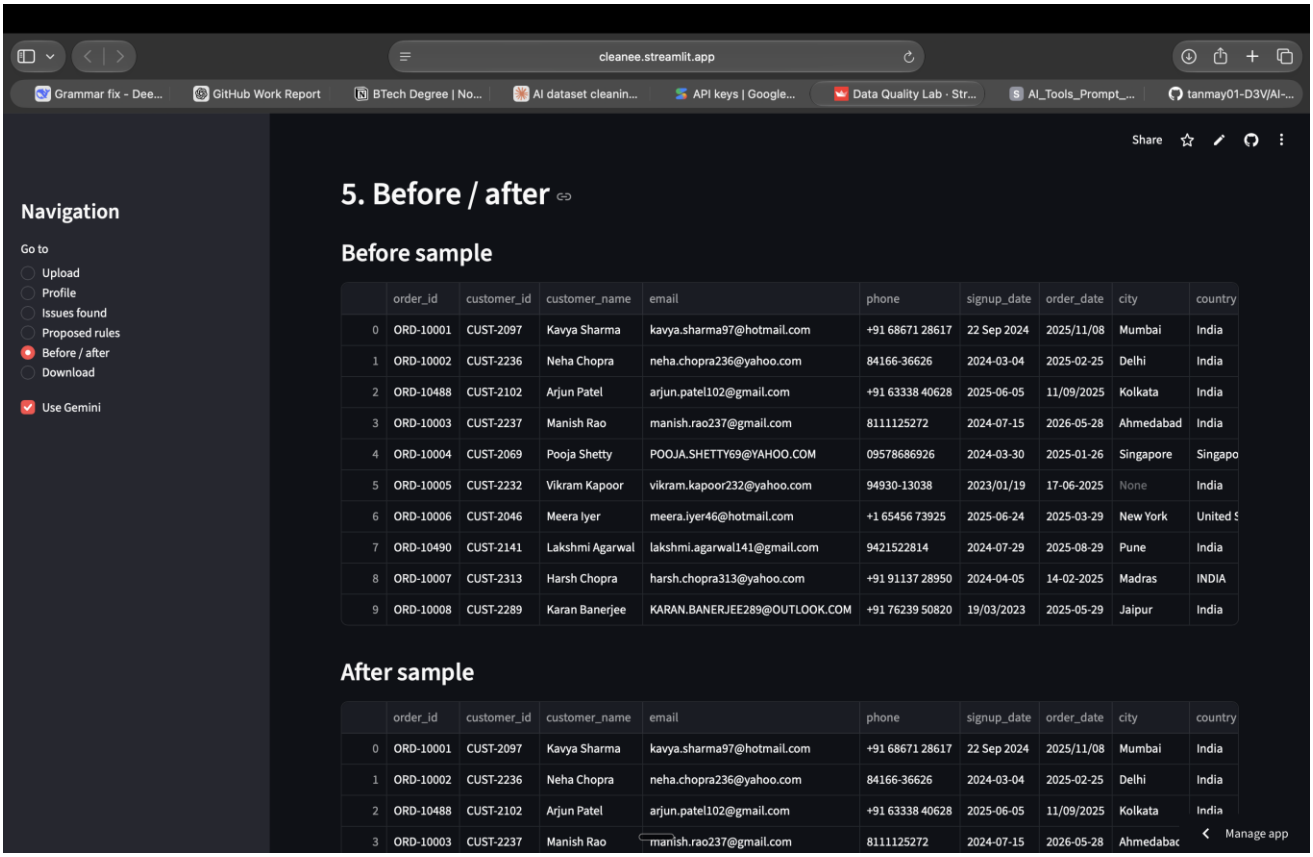


Figure 29: *Before and after preview of the cleaned data*

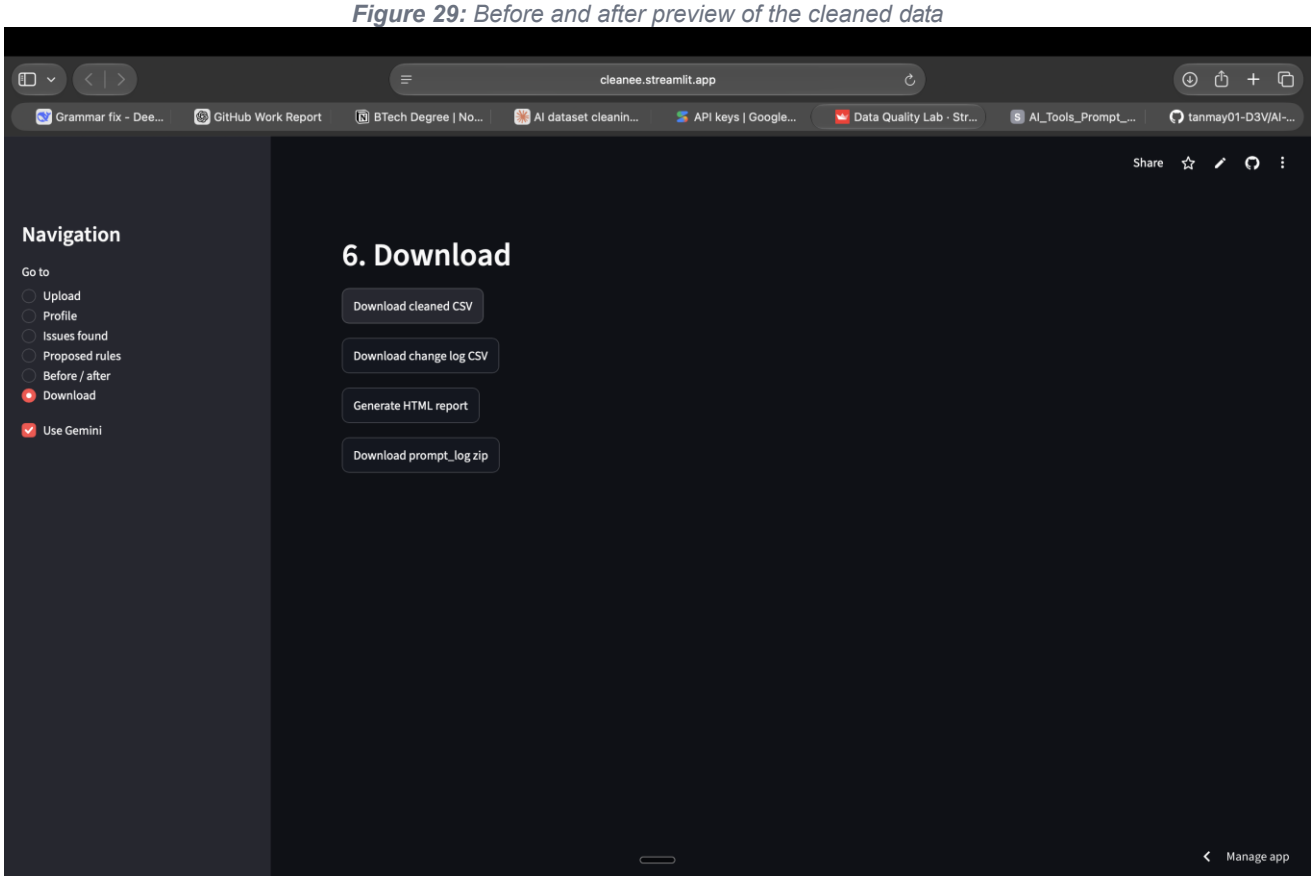


Figure 30: *Download page: cleaned CSV, report, change log, prompt log*

7. Challenges and Limitations

Challenge	How it was handled
Free-tier rate limits (HTTP 429)	Disk cache, retry with backoff, a second model id, fewer than 40 calls per file
Gemini returning invalid JSON or prose	JSON mode with a Pydantic schema, one retry, then a rule-only fallback
Wrong category merges (e.g. Delhi vs New Delhi)	needs_review list and a user accept or reject step; code never applies an unapproved rule
Ambiguous dates such as 04/05/2025	Day-first assumption stated in the interface and the report
Fuzzy matching removing a real, different customer	Near-duplicates are shown for confirmation and never auto-dropped
Privacy on the free tier	Only synthetic data is sent to the API
Gemini API returned 403: free tier not available for the project	The Google account in use was an organization account. The key was created under a personal account and the smoke test then returned OK.
Project folder was inside iCloud Drive	Moved to a normal local folder and recreated the virtual environment, so syncing cannot corrupt the git history or copy the key file.
The coding assistant opened the answer-key files while building the loader	Added a rule to the prompts and to Agent.md: never open the answer-key files; the evaluation reads them only when it runs.
Prompt text was pasted into the coding assistant's chat, so it was never run against Gemini	Used the explicit instruction 'Do not answer this prompt; edit cleaner/prompts.py' and ran every version through the prompt lab.
Script failed with ModuleNotFoundError for the cleaner package	Added the project root to the Python path at the top of the script.
Prompt lab crashed when writing its log (invalid append argument)	Replaced it with opening the log file in append mode.
Coding assistant token limit was reached part-way through the build	Switched from Antigravity to OpenCode and kept the shared instructions in Agent.md so work could continue.

Limitations. One file at a time; dates assumed day-first; accuracy depends on the free Gemini model available; no fix is attempted where the correct value cannot be known, such as a total that disagrees with quantity and price.

8. Conclusion and Future Work

8.1 Conclusion

The project shows that an AI tool is most dependable when responsibilities are separated: code does detection and editing, and the language model supplies judgment about meaning, always as validated JSON and always behind a human approval step. Prompt engineering made the difference between unusable and usable model output, and the logged versions document how each prompt improved.

What I learned: giving each prompt one narrow job, a fixed JSON output and an approval step made the AI output safe to use, and logging every prompt version with its failure made each improvement easy to explain. Final measured results are in the Testing and Results section.

8.2 Future work

- Support several files and joins between them.
- Learn column rules from a user's earlier corrections.
- Add a local model option so no data leaves the machine.
- Handle Excel and JSON inputs.
- Add statistical outlier methods beyond the IQR rule.

References

[1] Pandera and Great Expectations discussion. github.com/unionai-oss/pandera/discussions/598

[2] Posit. Python validation libraries survey, 2025. opensource.posit.co/blog/2025-06-04_validation-libs-2025

[3] A Survey of LLM x DATA. arXiv:2505.18458

[4] LLMClean: Context-Aware Tabular Data Cleaning via LLM-Generated OFDs. arXiv:2404.18681

[5] AutoDCWorkflow: LLM-based Data Cleaning Workflow Auto-Generation and Benchmark. arXiv:2412.06724

[6] Can LLMs Automate Data Cleaning? promptlayer.com/research-papers/can-llms-automate-data-cleaning

[7] Data Science with LLMs: Cleaning. third-bit.com/dsllm/clean

[8] RapidFuzz string matching library. Fuzzy duplicate tutorial: mljar.com/tutorials/find-fuzzy-duplicates-excel

[9] Google Gen AI Python SDK and Gemini API structured output documentation. ai.google.dev/gemini-api/docs

[10] Google. Gemini API troubleshooting (project unavailable error). ai.google.dev/gemini-api/docs/troubleshooting

Appendix A: Build prompt log

One row per build prompt. Keep the full prompt text in prompt_log/build_prompts.md in the repository.

ID	Prompt (core ask)	AI returned	Fixed by hand	Commit
B1	Read the CSV as text; blanks and fake nulls become null; profile each column; include tests.	loader.py, profiler.py	Nothing	40c9673
B2	Four detector modules for missing values, duplicates, formats and invalid values; no LLM calls; one test per check.	missing.py, duplicates.py, formats.py, invalid.py	Added an introduction to what we are building (context)	10a23b3
B3	Gemini client: JSON mode, retry on 429, disk cache, saved request and response logs, fake client for tests.	llm.py	Model-not-accepted error was an account problem, not a code problem	069338f
B4	Meta-prompt: explain why a prompt failed and propose a new version with the smallest change.	prompts.py	Nothing	-
B5	Rules engine returning the cleaned data and a change log; actions for dates, trimming, case, mappings, phones, money.	rules.py	Centralised common instructions in Agent.md	1936d21
B6	Pandera schema and validation reporting all failures at once.	schema.py	Antigravity tokens were exhausted; switched to OpenCode	5337f4c
B7	Self-contained Jinja2 HTML validation report.	report.py, templates/report.html.j2	Antigravity tokens were exhausted; switched to OpenCode	a42cf80
B8	Streamlit app: upload, profile, issues, rule review, before and after, downloads.	app.py	Signed in to Streamlit Cloud so the project can be deployed	b2c8f52

Appendix B: Links and files

Item	Location
GitHub repository	https://github.com/tanmay01-D3V/AI-Tool_mini_project
Deployment link	Live link
Dataset files	orders_dirty.csv, orders_ground_truth.csv, planted_issues.csv, answer_key_summary.json
Dataset generator	make_dirty_data.py (seed 42)
Prompt logs	prompt_log/build_prompts.md, prompt_log/tool_prompts.md, prompt_log/runs/